

CHURNCAST: AN MLOPS PIPELINE FOR SUBSCRIBER CHURN PREDICTION

A PROJECT REPORT

Submitted by

R SASIKIRAN

24AD0267

M A SANJAY KANTH

24AD0262

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CHENNAI INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to Anna University, Chennai & Recognized by AICTE New Delhi)

CHENNAI - 600 069

SEPTEMBER 2026

CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

(Affiliated to Anna University, Chennai & AICTE New Delhi)

CHENNAI - 600 069

BONAFIDE CERTIFICATE

Certified that this project report “**CHURNCast: AN MLOPS PIPELINE FOR SUBSCRIBER CHURN PREDICTION**” is the bonafide work of **R SASIKIRAN (24AD0267)** and **M A SANJAY KANTH (24AD0262)** who carried out the project work under my supervision.

Signature

HEAD OF THE DEPARTMENT
Department of Artificial Intelligence
and Data Science,
Chennai Institute of Technology,
Kundrathur, Chennai-600069.

Signature

Dr. A. ARAVIND, M.Tech.
PROJECT SUPERVISOR
Assistant Professor,
Department of Artificial Intelligence
and Data Science,
Chennai Institute of Technology,
Kundrathur, Chennai-600069.

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project report entitled “**CHURNCAS**T: AN **MLOPS PIPELINE FOR SUBSCRIBER CHURN PREDICTION**” submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science is a record of the original work carried out by us under the supervision of **Dr. A. ARAVIND, M.Tech., Assistant Professor**, Department of Artificial Intelligence and Data Science, Chennai Institute of Technology.

We further declare that the experimental results presented in this report were produced by executing the implementation described herein, and that every reported figure is traceable to an artifact generated by the system. Components that were implemented but not verified at runtime are identified as such in the relevant sections. The work reported here has not formed the basis for the award of any other degree or diploma.

R SASIKIRAN

24AD0267

M A SANJAY KANTH

24AD0262

Place: Chennai

Date: September 2026

ACKNOWLEDGEMENT

We express our gratitude to our Chairman Shri. P. SRIRAM and all trust members of Chennai Institute of Technology for providing the facility and opportunity to do this project as a part of our undergraduate course.

We are grateful to our Principal Dr. A. RAMESH M.E, Ph.D, for providing us the facility and encouragement during the course of our work.

We would like to extend our thanks to our Dean, Dr. V. SRINIVASA RAO M.Tech, PhD, for providing valuable suggestions throughout this project.

We sincerely thank our Head of the Department, Department of Artificial Intelligence and Data Science, for having provided us with valuable guidance, resources and timely suggestions throughout our work.

We sincerely thank our Project Supervisor and Project Coordinator, Dr. A. Aravind, M.Tech., Assistant Professor, Department of Artificial Intelligence and Data Science, whose guidance shaped both the technical direction of this work and the discipline of reporting only what could be demonstrated.

We wish to extend our sincere thanks to all the Faculty Members of the Department of Artificial Intelligence and Data Science for their valuable suggestions and kind cooperation towards the successful completion of our project.

We also acknowledge the help received from the Lab Instructors of the Department of Artificial Intelligence and Data Science for their assistance during the implementation and testing of this system.

ABSTRACT

Subscription-based entertainment and application services derive recurring revenue from active memberships, which makes subscriber attrition a direct and continuous threat to business viability. Because acquiring a replacement subscriber is substantially more expensive than retaining an existing one, identifying subscribers who are likely to lapse before their membership expires has become a central analytical requirement for streaming and application providers. This project presents ChurnCast, an end-to-end machine learning and Machine Learning Operations (MLOps) pipeline that estimates the probability that an active subscriber will fail to renew within thirty days of their membership expiration date, and exposes that estimate through a prediction service and a web interface in which an analyst enters only a member identifier.

The central technical concern of the work is data leakage. The churn label is defined relative to each subscriber's own membership expiration date, and the renewal transaction that determines the label occurs after that date. Any feature computed from records dated after the observation cutoff therefore encodes the outcome rather than evidence about it. An earlier iteration of this project, built on the original KKBOX extracts, applied a single global cutoff that fell inside the label period and reported an inflated discriminative score; a controlled experiment holding the subscriber population and labels fixed while retracting the feature window established that a substantial part of that score was attributable to contamination rather than to behavioural signal. That finding is reported in this work as a design lesson, and it directly motivates the leakage-aware feature construction and the strictly temporal evaluation protocol adopted in the final system.

The reported experimental results correspond to the reproducible synthetic KKBOX-like prototype dataset used for validating the complete MLOps workflow. The dataset comprises 7,500 subscribers with transaction and listening histories, from which 27 leakage-aware features are derived against each subscriber's own observation cutoff. Subscribers are partitioned into monthly cohorts by that cutoff, giving a genuine temporal split in which earlier cohorts train the model, a later cohort selects between candidates, and the latest cohort is scored exactly once. A calibrated Logistic Regression baseline and a calibrated gradient boosted tree candidate were compared; Logistic Regression was selected on validation ROC-

AUC of 0.7835 and achieved 0.7957 ROC-AUC, 0.6904 PR-AUC, 0.7951 accuracy, 0.7309 precision, 0.4579 recall and 0.5630 F1-score on the held-out cohort.

Every training run is tracked in MLflow with its parameters, metrics and artifacts, and the selected model is promoted through the MLflow Model Registry under a comparison gate that rejects candidates failing to improve on the incumbent. A FastAPI service resolves the registered model at start-up and serves member-identifier predictions, which a Next.js interface consumes. Data drift monitoring using Evidently reports no drift between cohorts of the same generative process and detects six of eight monitored features under a deliberately injected shift. Automated retraining is implemented as an Airflow directed acyclic graph that calls the same reusable pipeline functions, and containerised deployment is configured through Docker Compose; the runtime status of both is stated explicitly in the relevant sections. A suite of 74 automated tests covers the pipeline, including explicit guards that assert post-cutoff records cannot influence any feature.

KEYWORDS: Subscriber Churn, MLOps, Data Leakage, Temporal Validation, Feature Engineering, MLflow, Model Registry, FastAPI, Airflow, Evidently, Docker, Next.js, Customer Retention

TABLE OF CONTENTS

INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Motivation.....	2
1.4 Objectives	3
1.5 Scope.....	4
1.6 Contributions.....	4
1.7 Organization of the Report.....	5
LITERATURE SURVEY	6
2.1 Customer Churn Prediction.....	6
2.2 Supervised Classification and Class Imbalance.....	6
2.3 Ensemble Methods and Gradient Boosting.....	7
2.4 Probability Calibration.....	7
2.5 Data Leakage in Predictive Modelling	7
2.6 Temporal Validation	8
2.7 Behavioural Feature Engineering for Subscription Services	8
2.8 MLOps, Experiment Tracking and Model Registry	8
2.9 Model Serving and Workflow Orchestration.....	9
2.10 Data Drift Monitoring	9
2.11 Summary of the Survey	9
SYSTEM ANALYSIS AND REQUIREMENTS	11
3.1 Existing System	11
3.2 Limitations of Existing System.....	11
3.3 Proposed System.....	12

3.4 Functional Requirements	12
3.5 Non-Functional Requirements	13
3.6 Hardware Requirements.....	14
3.7 Software Requirements.....	14
3.8 Feasibility Analysis.....	15
SYSTEM DESIGN	16
4.1 Overall Architecture.....	16
4.2 Data Flow	16
4.3 Data Ingestion Architecture	17
4.4 Data Validation	17
4.5 Leakage-Aware Feature Engineering	18
4.6 Temporal Dataset Splitting	19
4.7 Model Training	19
4.8 MLflow Architecture	20
4.9 Model Registry.....	20
4.10 FastAPI Serving Architecture	21
4.11 Member-ID Prediction Flow	22
4.12 Frontend Architecture	23
4.13 Airflow Architecture.....	23
4.14 Evidently Monitoring Architecture.....	24
4.15 Docker Architecture.....	24
4.16 Storage and Artifact Design.....	25
IMPLEMENTATION.....	26
5.1 Development Environment	26
5.2 Dataset Generation and Data Preparation	26

5.2.1 Rationale for a Synthetic Prototype Dataset	26
5.2.2 Composition of the Synthetic Dataset.....	26
5.2.3 Relationship to the Original KKBOX Data	27
5.3 Feature Engineering	28
5.4 Model Development.....	29
5.5 Model Evaluation.....	29
5.6 MLflow Tracking.....	30
5.7 Model Registry.....	30
5.8 FastAPI Implementation	30
5.9 Member-ID Prediction	31
5.10 Next.js Frontend.....	31
5.11 Airflow DAG	31
5.12 Evidently Drift Monitoring	32
5.13 Docker Configuration	32
5.14 Testing.....	32
5.15 Debugging and Engineering Fixes.....	33
RESULTS AND DISCUSSION	35
6.1 Dataset Summary	35
6.2 Feature Summary	35
6.3 Model Performance.....	36
6.4 ROC-AUC.....	37
6.5 PR-AUC	37
6.6 Accuracy	37
6.7 Precision.....	37
6.8 Recall	37

6.9 F1 Score	38
6.10 Confusion Matrix	38
6.11 ROC Curve.....	39
6.12 Precision-Recall Curve	40
6.13 MLflow Results	41
6.14 Model Registry Results.....	42
6.15 Member-ID Prediction Results	42
6.16 Frontend Results	43
6.17 Drift Monitoring Results.....	43
6.18 Testing Results.....	44
6.19 Pipeline Reproducibility	45
6.20 Discussion and Limitations.....	45
6.20.1 Interpretation of the Results.....	45
6.20.2 Limitations of the Data	45
6.20.3 Limitations of the Model	46
6.20.4 Components Not Verified at Runtime	46
CONCLUSION AND FUTURE ENHANCEMENTS	48
7.1 Conclusion	48
7.2 Key Contributions.....	48
7.3 Limitations	49
7.4 Future Enhancements.....	50
Appendix A - Project Directory Structure	54
Appendix B - API Examples.....	55
B.1 Health Check	55
B.2 Model Information	55

B.3 Member-ID Prediction	55
B.4 Unknown Member Identifier.....	55
B.5 Invocation Examples	55
Appendix C - Configuration	57
Appendix D - Test Results.....	58
Appendix E - Sample Predictions	59
Appendix F - Important Code Snippets	60
F.1 Leakage Guard in Feature Engineering	60
F.2 Temporal Split	60
F.3 Nested Calibration Pipeline	60
F.4 Categorical Normalisation (Train/Serve Consistency).....	60
F.5 Member-ID Prediction Endpoint	60
F.6 Airflow DAG Definition.....	61
Abstract	64
I. Introduction	64
II. Method	64
III. Leakage Diagnosis	65
IV. Results.....	65
V. Discussion	66
VI. Conclusion	66
References.....	66

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Scope of the ChurnCast System	
4.1	Overall ChurnCast MLOps Architecture	
4.2	Data Processing Pipeline	
4.3	Leakage-Aware Feature Engineering Flow	
4.4	Temporal Train / Validation / Test Split	
4.5	MLflow Experiment Tracking and Model Registry Flow	
4.6	FastAPI Prediction Architecture	
4.7	Member-ID Prediction Flow	
4.8	Frontend to API to Model Prediction Flow	
4.9	Airflow Retraining DAG	
4.10	Evidently Drift Monitoring Flow	
4.11	Docker Compose Architecture	
6.1	Confusion Matrix on the Held-Out Test Cohort	
6.2	ROC Curves for Validation and Test Evaluation	
6.3	Precision-Recall Curves for Validation and Test Evaluation	

LIST OF TABLES

Table No.	Title	Page No.
1.1	Summary of Project Contributions	
3.1	Limitations of the Existing Approach	
3.2	Functional Requirements	
3.3	Non-Functional Requirements	
3.4	Hardware Requirements	
3.5	Software Requirements and Verified Versions	
3.6	Feasibility Analysis	
4.1	Temporal Cohort Allocation	
4.2	API Endpoint Specification	
4.3	Risk Level Thresholds	
5.1	Development Environment	
5.2	Synthetic Prototype Dataset Composition	
5.3	Preserved Original KKBOX Extracts	
5.4	Engineered Feature Groups	
5.5	Model Hyperparameters	
5.6	Test Suite Composition	
5.7	Engineering Defects Identified and Resolved	
6.1	Dataset Summary	
6.2	Validation Performance of Candidate Models	
6.3	Held-Out Test Performance of the Selected Model	
6.4	Per-Class Classification Report on the Test Cohort	
6.5	Confusion Matrix Values	
6.6	MLflow Tracked Runs and Registry State	
6.7	Member-ID Prediction Results	
6.8	Data Drift Monitoring Results	
6.9	Test Execution Summary	
6.10	Component Implementation and Verification Status	
7.1	Limitations of the Present Work	
7.2	Future Enhancements	
A.1	Program Outcome Attainment	
A.2	Program Specific Outcome Attainment	

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AUC	Area Under the Curve
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
EDA	Exploratory Data Analysis
ETL	Extract, Transform, Load
F1	Harmonic Mean of Precision and Recall
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KKBOX	KKBOX Music Streaming Service (WSDM Cup churn dataset provider)
ML	Machine Learning
MLOps	Machine Learning Operations
msno	Member Serial Number (anonymised subscriber identifier)
PR-AUC	Area Under the Precision-Recall Curve
REST	Representational State Transfer
RFM	Recency, Frequency, Monetary
ROC	Receiver Operating Characteristic
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SQLite	Embedded Structured Query Language Database Engine
UI	User Interface
WSDM	Web Search and Data Mining (ACM Conference)
WSL	Windows Subsystem for Linux
XGBoost	Extreme Gradient Boosting
YAML	YAML Ain't Markup Language

CHAPTER 1

INTRODUCTION

1.1 Background

Subscription services have become the dominant commercial model in digital entertainment. Music streaming platforms, video services and mobile applications increasingly derive revenue from recurring memberships rather than from one-time purchases. Under this model the value of a customer is determined not by a single transaction but by the number of consecutive billing cycles over which the relationship persists. The rate at which subscribers discontinue their memberships, referred to as the churn rate, is consequently among the most closely monitored indicators in the sector.

The economic asymmetry underlying churn is well established. Acquiring a subscriber requires marketing expenditure, promotional discounting and onboarding effort, whereas retaining an existing subscriber generally requires only continued service quality and occasional targeted intervention. A service losing a substantial fraction of its base each year must therefore spend continuously simply to replace lost revenue before achieving net growth.

Churn in subscription services is frequently silent. Automatic renewal makes continuation the default behaviour, so a subscriber who becomes dissatisfied often does not complain; they disable automatic renewal or allow the membership to lapse. The behavioural precursors of that decision, however, are observable in operational data well before the subscription ends. Declining listening activity, a reduction in session frequency, a change in payment behaviour or a history of prior cancellations are all recorded by the platform. This observability is what makes predictive modelling worthwhile: the information required to anticipate a lapse exists before the lapse occurs.

1.2 Problem Statement

The problem addressed by this project is the estimation, for each active subscriber, of the probability that the subscriber will fail to renew their membership within thirty days of its expiration date. Formally this is a binary classification task in which the positive class

denotes churn. The operational requirement is not merely accurate labelling but calibrated probabilities that permit subscribers to be ranked by risk, so that a retention team with finite outreach capacity can prioritise effectively.

Two properties make the task substantially harder than conventional classification. The first is class imbalance: in most subscription businesses the majority of subscribers renew, so a model predicting the negative class unconditionally attains high accuracy while being operationally useless. The second, and the more consequential for this project, is temporal structure. The churn label is not an attribute of the subscriber at prediction time; it is determined by events occurring after the membership expiration date. Any feature computed from records inside that future window risks encoding the answer directly, producing a model that appears highly accurate during offline evaluation and fails on deployment.

This project therefore treats the temporal boundary between the observation window and the label period as a first-class design constraint rather than an implementation detail, and treats its verification as an explicit testing obligation.

1.3 Motivation

The motivation for this work arose directly from a defect encountered during its development. An earlier iteration of the system, constructed on the original KKBOX extracts, applied a single global observation cutoff to every subscriber. That cutoff fell inside the period during which the renewal decision was recorded, with the consequence that the features describing recent transaction activity encoded whether the renewal had already occurred. The resulting model reported a high discriminative score that did not reflect any ability to anticipate churn.

The defect was not visible in any conventional evaluation metric. It produced no exception, no failing assertion and no anomalous log entry; the only symptom was a score higher than the problem warranted. It was identified through deliberate diagnostic work: inspection of conditional outcome distributions revealed a step discontinuity at the boundary of the label window rather than the smooth gradient a genuine behavioural feature would produce, and a controlled experiment holding the subscriber population and labels fixed while retracting the

feature window showed the score declining monotonically as access to the label period was withdrawn.

That episode motivates the three principles governing the present system: the observation cutoff is declared once and applied uniformly to every data source; evaluation partitions are ordered in time rather than sampled at random; and the absence of leakage is asserted by automated tests rather than assumed. The earlier inflated result is reported in this document as a validation lesson and is not presented as a performance outcome of this project.

1.4 Objectives

1. To construct a reproducible dataset that reflects the structure and semantics of subscription churn data, with subscriber-specific observation cutoffs and a label determined strictly after those cutoffs.
2. To implement a leakage-aware feature engineering layer that derives recency, frequency, engagement, subscription and demographic features exclusively from records dated on or before each subscriber's own observation cutoff.
3. To implement a temporal train, validation and test partition based on subscriber cohorts, such that evaluation always looks forward in time.
4. To train and calibrate a Logistic Regression baseline and a gradient boosted tree candidate, and to select between them using validation performance alone.
5. To evaluate the selected model exactly once on a held-out later cohort and report ROC-AUC, PR-AUC, accuracy, precision, recall, F1-score and the confusion matrix.
6. To track every experiment in MLflow and manage model promotion through the MLflow Model Registry under a comparison gate.
7. To serve the registered model through a FastAPI application supporting prediction by member identifier, and to provide a web interface that consumes it.
8. To implement data drift monitoring using Evidently, automated retraining orchestration using Airflow, and containerised deployment using Docker Compose.
9. To validate the system through an automated test suite that includes explicit assertions against data leakage.

1.5 Scope

The scope of this work covers the complete machine learning lifecycle for subscriber churn prediction: dataset preparation, leakage-aware feature engineering, temporal evaluation, model training and calibration, experiment tracking, registry-based model promotion, API serving, a web interface supporting member-identifier prediction, drift monitoring, retraining orchestration and containerisation.

The experimental results reported in Chapter 6 correspond to the reproducible synthetic KKBOX-like prototype dataset described in Section 5.2. The original KKBOX extracts are retained in the repository as reference data, and the relationship between the two is set out in Section 5.2.3. No claim is made in this report regarding performance on the official KKBOX benchmark.

The following lie outside the present scope: real-time streaming ingestion, integration with a live customer relationship management platform, automated execution of retention offers, causal estimation of intervention effectiveness, and deployment to a cloud environment serving live subscriber traffic.

Figure 1.1 Scope of the ChurnCast System

IN SCOPE	OUT OF SCOPE
-----	-----
Dataset preparation and validation	Real-time streaming
ingestion	
Leakage-aware feature engineering	CRM platform integration
Temporal train / validation / test split	Automated retention offers
Model training, calibration, selection	Causal uplift estimation
MLflow tracking and model registry	Cloud production deployment
FastAPI serving, member-ID prediction	Live subscriber traffic
Next.js frontend	
Evidently drift monitoring	
Airflow retraining orchestration	
Docker Compose configuration	
Automated test suite	

1.6 Contributions

Table 1.1 Summary of Project Contributions

No.	Contribution	Where Demonstrated
1	A leakage-aware feature engineering layer in which the observation cutoff is per-subscriber, declared once and applied uniformly to every source table	Sections 4.5, 5.3

2	Automated tests that assert post-cutoff transactions and listening records cannot alter any computed feature value	Sections 5.14, 6.18
3	A genuinely temporal evaluation protocol based on subscriber cohorts, with model selection on validation and a single scoring pass on a later cohort	Sections 4.6, 6.3
4	An empirical account of a target leakage defect, its detection method and its measured effect, reported as a validation lesson	Sections 1.3, 5.15
5	A registry promotion gate that rejects candidate models failing to improve on the incumbent, observed rejecting a tied candidate	Sections 4.9, 6.14
6	A member-identifier prediction service and web interface in which feature retrieval occurs server-side, eliminating client-side feature duplication	Sections 4.11, 6.15, 6.16
7	Identification and correction of a train/serve representation skew that silently disabled two categorical features at inference	Section 5.15
8	A reproducible pipeline verified to regenerate identical metrics from a clean state under a fixed seed	Section 6.19

1.7 Organization of the Report

- **Chapter 2** surveys the literature on churn prediction, supervised classification, temporal validation, data leakage, behavioural feature engineering and MLOps.
- **Chapter 3** analyses the existing approach, states its limitations, and specifies the functional, non-functional, hardware and software requirements of the proposed system together with a feasibility analysis.
- **Chapter 4** presents the system design, covering the overall architecture and each constituent layer from ingestion through to containerised deployment.
- **Chapter 5** describes the implementation of each module, the test suite, and the engineering defects encountered and resolved during development.
- **Chapter 6** reports the experimental results, including model performance, registry state, prediction behaviour, drift monitoring, test execution and reproducibility, together with a discussion of limitations.
- **Chapter 7** concludes the work, restates its contributions and limitations, and identifies future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Customer Churn Prediction

Research on customer churn prediction spans more than two decades, originating largely in the telecommunications sector where usage records provided an early source of behavioural data. Verbeke et al. [1] examined the gap between statistical accuracy and business usefulness in churn models, arguing that comprehensibility and operational justifiability matter alongside predictive power. In subsequent work [2] the same group advanced a profit-driven evaluation perspective, observing that the value of a churn model depends on the cost structure of the retention campaign it informs rather than on discrimination alone. Both observations motivate the retention of an interpretable baseline model in the present system, and the reporting of calibrated probabilities rather than ranks alone.

2.2 Supervised Classification and Class Imbalance

Churn prediction is conventionally posed as supervised binary classification. Logistic Regression remains the standard baseline because its coefficients admit direct interpretation as contributions to the log-odds of the outcome, a property valued where predictions must be explained. Hastie et al. [3] provide the statistical treatment of regularised linear classification adopted here.

Because churners typically constitute a minority of the subscriber base, accuracy is an inappropriate primary metric and some form of imbalance handling is required. Chawla et al. [4] introduced the Synthetic Minority Over-sampling Technique, which remains widely used. Its application to temporally ordered data requires caution, however, since synthetic examples interpolated between observations may violate the separation between training and evaluation periods. Idris et al. [5] examined resampling in combination with feature selection for telecommunications churn. For the reasons set out in Section 5.4, the present work prefers class weighting within the learning algorithm to synthetic resampling.

2.3 Ensemble Methods and Gradient Boosting

Ensemble methods dominate reported results on tabular churn data. Breiman [6] established bagged decision tree ensembles as a robust default. Friedman [7] formulated gradient boosting as stagewise additive modelling, in which each successive learner is fitted to the gradient of the loss with respect to the current ensemble prediction. Chen and Guestrin [8] contributed the scalable XGBoost implementation used as the candidate model in this project, and Ke et al. [9] subsequently introduced histogram-based splitting in LightGBM for improved training efficiency.

The literature also records the principal weakness of boosted ensembles in this setting. Because boosting exploits any available signal, a contaminated feature is absorbed more completely by a boosted ensemble than by a linear model, which makes leakage both more damaging and harder to detect from metrics alone.

2.4 Probability Calibration

The reliability of predicted probabilities, as distinct from ranking quality, has been addressed by Platt [10] through sigmoid scaling of classifier outputs and by Zadrozny and Elkan [11] through isotonic regression. Niculescu-Mizil and Caruana [12] compared these approaches across learning algorithms and demonstrated that boosted trees in particular produce distorted probability estimates that benefit substantially from post-hoc calibration. Since operational prioritisation by expected value requires probabilities rather than ranks, both candidate models in this project are wrapped in a cross-validated calibration stage.

2.5 Data Leakage in Predictive Modelling

Kaufman et al. [13] provide the formal treatment of data leakage, defining it as the introduction of information about the prediction target that would not legitimately be available at prediction time, and documenting its prevalence in data mining competitions. Their analysis of leakage in temporally structured problems applies directly to the churn setting, where the label is determined by events occurring after the observation point. The authors note that leakage is characteristically invisible to standard evaluation, since the contaminated feature improves the measured score precisely because it encodes the outcome.

This observation is consistent with the experience reported in Section 1.3 and shapes the diagnostic procedure described in Section 5.15.

2.6 Temporal Validation

Bergmeir and Benitez [14] examined the validation of predictors on temporally ordered data and argued that evaluation partitions must respect temporal ordering if the resulting estimate is to describe forward generalisation. Randomly partitioning records drawn from a single period evaluates the model under a condition it will not encounter in deployment. This finding supports the cohort-based temporal split adopted in Section 4.6 in preference to random partitioning.

2.7 Behavioural Feature Engineering for Subscription Services

A consistent finding across the churn literature is that feature construction contributes more to predictive performance than the choice of learning algorithm. The Recency, Frequency and Monetary framework, originating in direct marketing, provides a durable organising structure: how recently a subscriber was active, how often they transact, and the monetary value of those transactions. In subscription services this is extended with engagement measures derived from consumption logs, such as active days, total consumption time and the ratio of completed to abandoned content. Features capturing the direction of change in engagement rather than its level are repeatedly reported among the stronger predictors, since disengagement is a process rather than an event. The feature groups described in Section 5.3 follow this structure.

2.8 MLOps, Experiment Tracking and Model Registry

Sculley et al. [15] characterised the accumulation of technical debt in machine learning systems, observing that machine learning code constitutes a small fraction of a deployed system and that the remainder, comprising data collection, verification, serving infrastructure and monitoring, gives rise to distinctive maintenance hazards. Breck et al. [16] translated these observations into a testable readiness rubric covering feature expectation testing, model quality validation and infrastructure monitoring. Amershi et al. [17] reported a case study of

machine learning engineering practice, identifying experiment management and model versioning as recurring difficulties.

Zaharia et al. [18] described MLflow, the platform adopted in this project for experiment tracking and model registry management. Its relevance here is practical rather than incidental: as recorded in Section 5.15, the availability of tracked metric history across runs was material to detecting the defects discussed in that section.

2.9 Model Serving and Workflow Orchestration

Model serving in this project uses FastAPI, which provides declarative request validation through Python type annotations and automatic generation of an OpenAPI specification. Workflow orchestration uses Apache Airflow, in which pipelines are expressed as directed acyclic graphs of discrete tasks with explicit dependencies, retry policies and execution history. Both choices follow the separation of concerns advocated by Sculley et al. [15]: the orchestration layer schedules and retries, while the computational logic remains in independently testable modules.

2.10 Data Drift Monitoring

A deployed model degrades when the distribution of its inputs diverges from that of its training data. Gama et al. [19] surveyed concept drift adaptation and distinguished changes in the input distribution from changes in the relationship between inputs and target, noting that the former is observable without ground truth whereas the latter is not. This distinction determines the monitoring design in Section 4.14: feature drift is computed on every scoring batch, whereas performance comparison requires labels that become available only after the prediction horizon has elapsed.

2.11 Summary of the Survey

The surveyed literature supports four design decisions adopted in this project. First, an interpretable baseline is retained alongside a higher-capacity model so that the gain attributable to model complexity is measurable. Second, probability calibration is applied so that outputs support expected-value prioritisation. Third, temporal ordering is respected in the evaluation partition. Fourth, leakage is treated as a condition to be tested for rather than

assumed absent, since the literature establishes both its prevalence and its invisibility to conventional metrics.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System

Churn analysis in subscription businesses is commonly performed retrospectively. Cohort retention curves and periodic dashboards describe how many subscribers were lost in a past period and, at an aggregate level, which segments they belonged to. Where predictive modelling is attempted, it is frequently carried out in notebook environments: a dataset is assembled, a classifier is fitted, a score is reported, and the resulting model is transferred to whatever consumes it as a serialised file.

An earlier iteration of the present project followed a comparable pattern. Features were computed over the available history using a single global cutoff chosen to maximise the number of subscribers possessing usable history; the evaluation partition was produced by random sampling; probabilities were reported without calibration assessment; and orchestration, monitoring and containerisation were absent. That iteration reported a high discriminative score which subsequent diagnosis attributed substantially to target leakage, as described in Section 1.3.

3.2 Limitations of Existing System

Table 3.1 Limitations of the Existing Approach

No.	Limitation	Consequence
1	Retrospective aggregate reporting only	No subscriber-level forward-looking risk estimate, so retention effort cannot be prioritised at the individual level
2	Observation cutoff not derived from the label definition	The feature window may overlap the label period, admitting information that encodes the outcome rather than predicting it
3	Random train/test partitioning	Evaluation does not reproduce the deployment condition of predicting forward in time, and therefore overstates generalisation
4	Absence of automated leakage checks	Contamination produces no error and no anomalous metric, so it can persist undetected through review
5	Uncalibrated probability outputs	Scores cannot be interpreted as risk levels and do not support expected-value ranking
6	Silent exclusion of subscribers lacking history	The modelled population diverges from the labelled population, introducing a sampling bias invisible to the reported metric

7	No experiment tracking or model versioning	Results cannot be traced to the configuration that produced them, and regressions are not detectable
8	No drift monitoring after deployment	Degradation is discovered only when downstream business metrics move
9	Manual, ad hoc retraining	The pipeline is not reproducible and retraining is error-prone

3.3 Proposed System

ChurnCast addresses these limitations through a layered pipeline in which each stage produces a versioned artifact consumed by the next. The observation cutoff is a per-subscriber property, declared once and applied uniformly to every source table before any aggregation occurs. Subscribers are assigned to monthly cohorts by that cutoff, and the cohorts are allocated whole to the training, validation and test partitions so that evaluation is ordered in time. Both candidate models are calibrated, and the selected model is registered under a gate that compares it against the incumbent.

The absence of leakage is asserted by automated tests which introduce post-cutoff records into a controlled fixture and require that no feature value changes. Subscribers lacking behavioural history are retained through left joins with explicit missingness indicators rather than excluded, and coverage statistics are logged on every feature build. Prediction is served by member identifier so that feature retrieval occurs server-side and cannot diverge between training and serving.

3.4 Functional Requirements

Table 3.2 Functional Requirements

ID	Requirement	Status
FR-01	Generate a reproducible dataset with subscriber-specific observation cutoffs and labels determined after those cutoffs	Implemented
FR-02	Validate source tables for schema conformance, key uniqueness and referential consistency	Implemented
FR-03	Derive recency, frequency, engagement, subscription and demographic features using only records dated on or before each cutoff	Implemented
FR-04	Retain every labelled subscriber in the feature table, with explicit indicators where behavioural history is absent	Implemented
FR-05	Partition subscribers into training, validation and test sets ordered in time by cohort	Implemented
FR-06	Train a Logistic Regression baseline and a gradient boosted tree candidate with probability calibration	Implemented
FR-07	Select the model on validation performance and score it once on the held-	Implemented

	out cohort	
FR-08	Record parameters, metrics, tags and artifacts for every run in MLflow	Implemented
FR-09	Register the selected model and reject candidates that do not improve on the incumbent	Implemented
FR-10	Serve predictions over HTTP with health and model information endpoints	Implemented
FR-11	Predict churn for a subscriber given only a member identifier, retrieving features server-side	Implemented
FR-12	Return a churn probability, predicted label, risk level and model version	Implemented
FR-13	Return HTTP 404 for an unknown identifier and HTTP 422 for a malformed one	Implemented
FR-14	Provide a web interface accepting a member identifier and displaying the prediction returned by the service	Implemented
FR-15	Generate data drift reports comparing a reference cohort against a current cohort	Implemented
FR-16	Orchestrate retraining as a scheduled workflow with retry handling	Implemented; runtime execution not verified
FR-17	Provide containerised deployment configuration for all services	Configured; runtime verification pending

3.5 Non-Functional Requirements

Table 3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Reproducibility	A fixed seed must regenerate an identical dataset, split and set of metrics from a clean state
NFR-02	Correctness	Features must be provably independent of post-cutoff records, asserted by automated tests
NFR-03	Consistency	Feature representation must be identical between training and serving
NFR-04	Traceability	Every reported metric must correspond to a generated artifact and a tracked run
NFR-05	Observability	Coverage, class balance and constant-feature conditions must be logged on every feature build
NFR-06	Maintainability	Pipeline logic must exist in one place and be shared by the script and the orchestrator
NFR-07	Robustness	The service must remain available and report its condition when no model can be loaded
NFR-08	Usability	Prediction must require only a member identifier, with no manual feature entry
NFR-09	Configurability	Paths, thresholds and endpoints must be configurable through environment variables
NFR-10	Portability	The system must run on a standard development machine without specialised hardware

3.6 Hardware Requirements

Table 3.4 Hardware Requirements

Component	Minimum	Used During Development
Processor	Dual-core x86-64	x86-64 multi-core
Memory	8 GB RAM	Sufficient for the 7,500-subscriber prototype; the full pipeline completes in approximately 60 seconds
Storage	5 GB free	Approximately 25 MB for the synthetic dataset and artifacts; the preserved original extracts occupy approximately 640 MB
Accelerator	Not required	No GPU used; all training is CPU-based
Network	Required only for dependency installation	No network access required at runtime; the dataset is generated locally

3.7 Software Requirements

The versions listed below are those present in the development environment at the time the reported results were generated.

Table 3.5 Software Requirements and Verified Versions

Component	Version	Role
Python	3.13.5	Primary implementation language
pandas	2.3.3	Tabular data processing and aggregation
NumPy	2.5.1	Numerical primitives
scikit-learn	1.9.0	Logistic Regression, preprocessing, calibration, metrics
XGBoost	3.3.0	Gradient boosted tree candidate model
matplotlib	3.11.0	Generation of evaluation figures
MLflow	3.14.0	Experiment tracking and model registry
Evidently	0.7.21	Data drift reporting
FastAPI	0.139.0	Prediction service
Pydantic	2.13.4	Request and response schema validation
Uvicorn	0.51.0	ASGI server
pytest	9.1.1	Automated test execution
Node.js	22.17.0	Frontend runtime
Next.js	16.3.5	Web interface framework
Apache Airflow	Not installed on the host	Retraining orchestration; no release supports Python 3.13, so the DAG targets the container environment
Docker Desktop	4.91.0 (engine not started)	Containerised deployment; see Section 6.20
Git	2.51.0	Version control
Ruff / Black	0.6+ / 24.0+	Linting and formatting

3.8 Feasibility Analysis

Table 3.6 Feasibility Analysis

Dimension	Assessment
Technical	Feasible and demonstrated. The complete pipeline from dataset generation through to web-based prediction executes on a standard development machine in approximately 60 seconds, and is covered by 74 automated tests. Two components, orchestration and containerisation, are implemented but were not verified at runtime for the environmental reasons recorded in Section 6.20.
Economic	Feasible. All components are open-source and were used under their respective licences. No cloud resources, paid services or specialised hardware were required. The principal cost is development effort.
Operational	Feasible for the intended use. Prediction requires only a member identifier, so the interface is usable by a retention analyst without knowledge of the underlying feature space. The registry-based promotion gate allows model updates without changing the serving configuration.
Schedule	Feasible. The work was completed within the project period. Two verification activities remain outstanding and are identified as deployment steps rather than development work.
Legal and ethical	The original KKBOX extracts are competition data subject to the terms accepted on download and are retained locally rather than redistributed. The dataset used for the reported experiments is synthetic and contains no personal data. All identifiers in the prototype are generated and refer to no real individual.

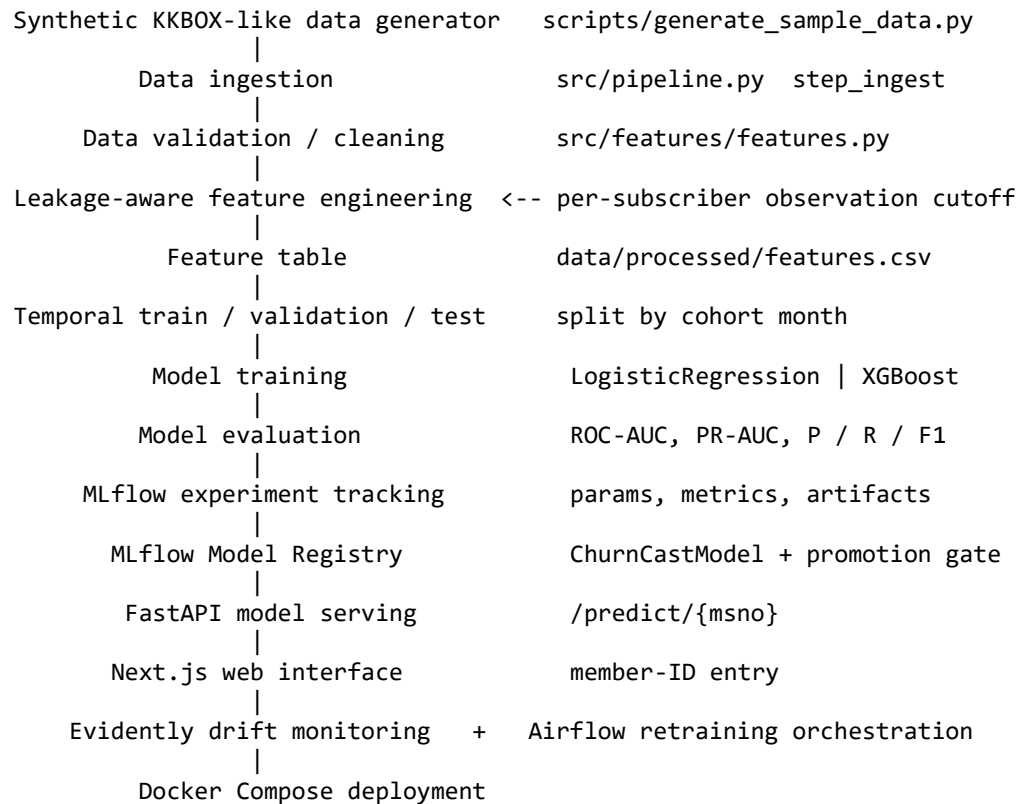
CHAPTER 4

SYSTEM DESIGN

4.1 Overall Architecture

ChurnCast is organised as a sequential pipeline of layers, each consuming the versioned output of its predecessor. The organising principle of the design is that the observation cutoff is a first-class property of the system rather than an incidental parameter: it is a per-subscriber value, declared once, applied uniformly to every source table, recorded with each experiment, and asserted by automated tests.

Figure 4.1 Overall ChurnCast MLOps Architecture

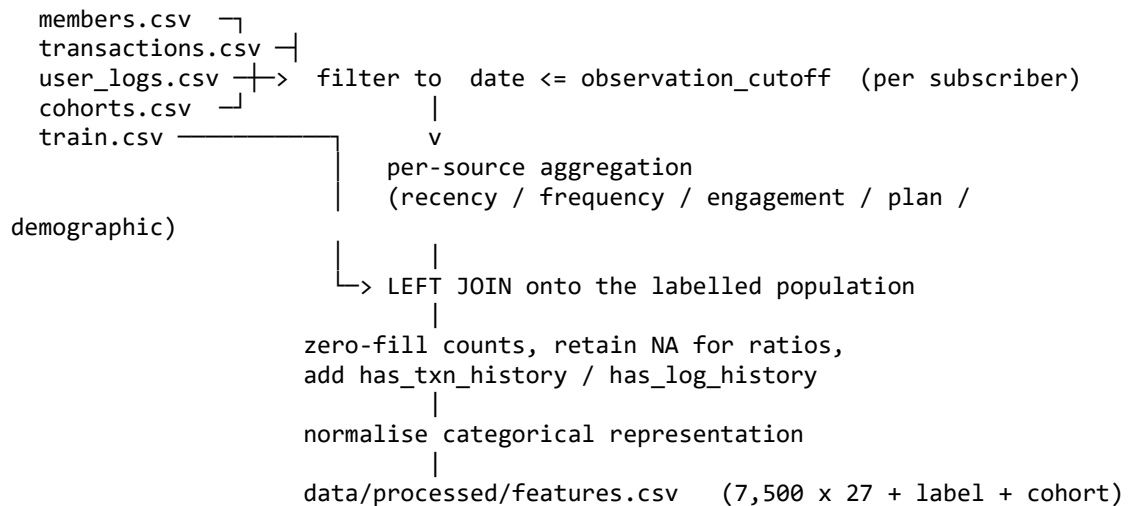


4.2 Data Flow

Data enters as four subscriber-keyed tables together with a cohort table supplying each subscriber's observation cutoff. Validation parses dates and confirms referential consistency. Feature engineering filters every source to the permitted window and reduces the event tables

to one row per subscriber. The resulting feature table is persisted, then partitioned by cohort and consumed by training. The fitted model is logged as an artifact, registered, and later resolved by the serving layer, which reads the same persisted feature table for identifier-based lookup.

Figure 4.2 Data Processing Pipeline



4.3 Data Ingestion Architecture

Ingestion is idempotent. If the expected files are already present in the raw data directory the generation step is skipped unless explicitly forced, and the existing row counts are reported. This allows the pipeline to be re-executed without unnecessarily regenerating the dataset, while a force flag guarantees a clean regeneration when reproducibility must be demonstrated from scratch.

The ingestion layer also preserves a distinction that matters for the integrity of the reported results. The original KKBOX extracts occupy a separate directory and are never read by the prototype pipeline; the synthetic dataset occupies the raw data directory and is the sole input to every experiment reported in Chapter 6. The input directory is configurable, so substituting the original extracts requires a configuration change rather than a code change.

4.4 Data Validation

Validation operates at two levels. Structural validation parses date fields with coercion so that malformed values become missing rather than halting the run, normalises categorical

identifiers to a single canonical representation, and confirms that subscriber-level tables have unique keys.

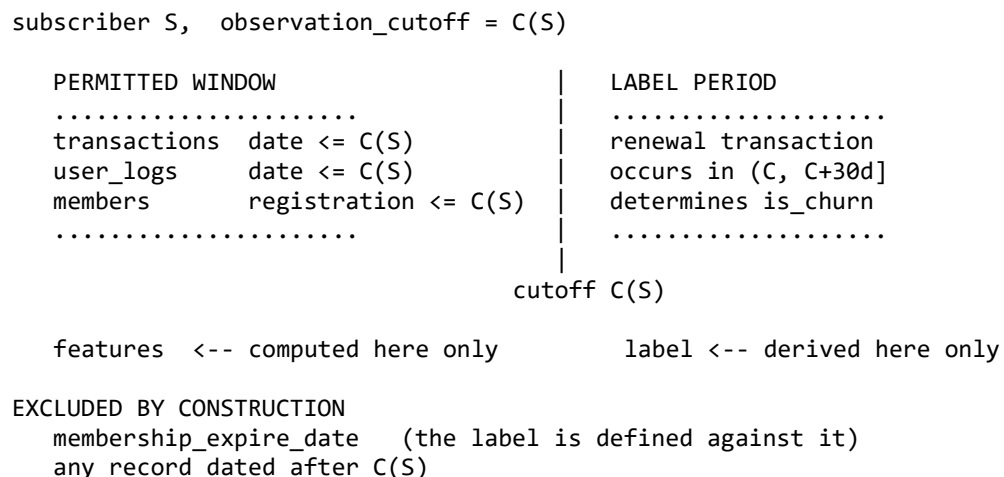
Statistical validation is emitted as diagnostics on every feature build. The layer reports the number of labelled subscribers, the overall class balance, the proportion possessing transaction and listening history before the cutoff, and the class balance within and outside each of those groups. A warning is raised for any feature exceeding fifty percent missingness and for any numerical feature that is constant, since a constant feature cannot contribute to the model and its presence indicates an upstream data fault. These diagnostics exist because a silent population loss in an earlier iteration was detectable only by comparing counts manually.

4.5 Leakage-Aware Feature Engineering

Each subscriber has an observation cutoff equal to the membership expiry date about which the churn question is asked. A feature may read only records dated on or before that subscriber's cutoff. The renewal transaction determining the label falls strictly after it, so a correct implementation cannot observe it.

Two categories of field are excluded from the feature space by construction. The membership expiry date itself is excluded because the label is defined against it, so any feature derived from it restates the question rather than providing evidence. Records dated after the cutoff are excluded by a filter applied to each source table before any aggregation occurs.

Figure 4.3 Leakage-Aware Feature Engineering Flow



ASSERTED BY TEST

adding a post-cutoff transaction -> no feature value changes

adding a post-cutoff log record -> no feature value changes

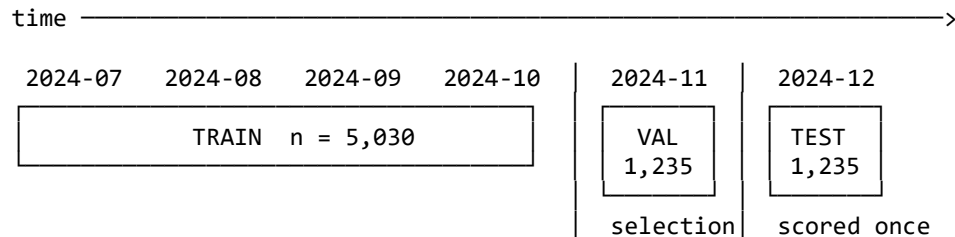
4.6 Temporal Dataset Splitting

Subscribers are assigned to monthly cohorts by their observation cutoff, and whole cohorts are allocated to the partitions. Earlier cohorts form the training set, a later cohort forms the validation set used for model selection, and the latest cohort forms the test set, which is scored exactly once. This reproduces the deployment condition in which a model trained on the past must predict the future.

Table 4.1 Temporal Cohort Allocation

Partition	Cohort months	Subscribers	Purpose
Training	2024-07, 2024-08, 2024-09, 2024-10	5,030	Model fitting
Validation	2024-11	1,235	Model selection and promotion gate comparison
Test	2024-12	1,235	Single held-out evaluation; never used for selection

Figure 4.4 Temporal Train / Validation / Test Split



Every validation cohort is later than every training cohort, and the test cohort is later still. Asserted by `test_temporal_split_is_ordered_in_time`.

4.7 Model Training

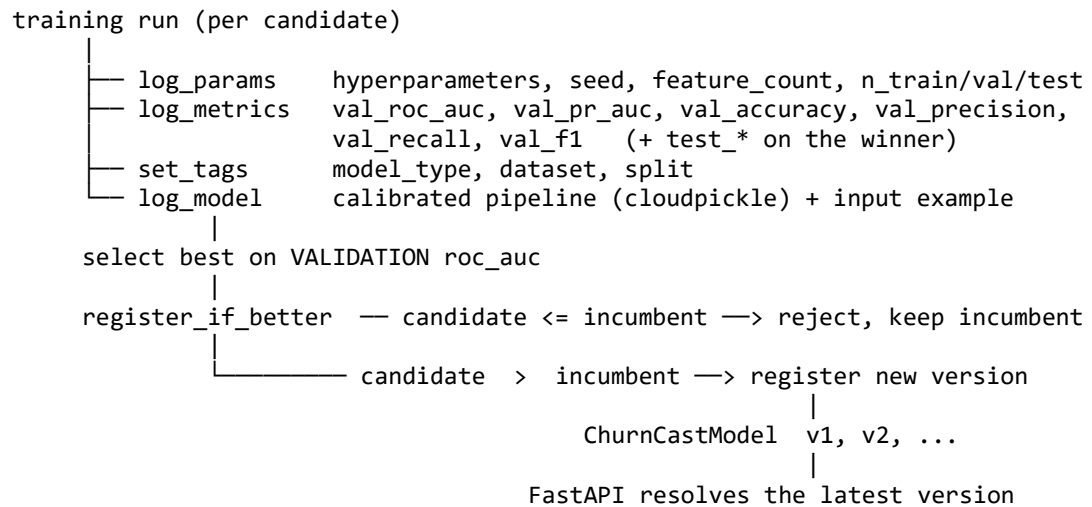
Preprocessing and estimation are bound into a single fitted object. Numerical features pass through median imputation and standardisation; categorical features pass through most-frequent imputation and one-hot encoding configured to ignore categories unseen during fitting. These transformers are composed into a column transformer placed inside a pipeline together with the estimator, and the entire pipeline is wrapped in a cross-validated calibration stage.

This nesting order is a deliberate correctness measure. Because the column transformer sits inside the pipeline, which sits inside the calibration wrapper, imputation statistics and scaling parameters are fitted separately within each calibration fold. No statistic derived from a validation fold can therefore influence the transformation applied to that fold, which eliminates preprocessing leakage independently of the target leakage question addressed in Section 4.5.

4.8 MLflow Architecture

Each training run is recorded as a tracked run within a named experiment. The run captures model hyperparameters, the random seed, the feature count, the sizes of all three partitions, the calibration configuration, the decision threshold, the six evaluation metrics, descriptive tags identifying the model type and the split strategy, and the serialised fitted pipeline together with a representative input example. Evaluation figures and the metrics document are attached to the winning run as artifacts.

Figure 4.5 MLflow Experiment Tracking and Model Registry Flow



4.9 Model Registry

The registry holds versions of a single registered model. Promotion is governed by a comparison gate: before registering, the candidate's validation ROC-AUC is compared against the best validation ROC-AUC among already-registered versions, obtained by resolving each version back to its originating run. A candidate that does not strictly improve

on the incumbent is rejected and the existing model is retained, with the decision recorded in the metrics document.

Comparison uses validation rather than test performance deliberately. Using the held-out cohort to decide promotion would convert it into a selection set and invalidate it as an estimate of forward generalisation.

4.10 FastAPI Serving Architecture

The serving layer resolves and loads the registered model once during the application lifespan hook rather than per request, and holds it in process state. A failure to resolve a model is caught and recorded rather than raised, so the process remains available and the health endpoint can report the condition; prediction requests then return a service-unavailable status rather than an unhandled exception.

Figure 4.6 FastAPI Prediction Architecture

```

application start-up (lifespan hook)
├── load feature table -> data/processed/features.csv
│   and compute per-column defaults for partial payloads
└── resolve registered model -> models:/ChurnCastModel/<latest>
    (failure is recorded, not raised; /health reports it)

request routing
GET /health                service + model availability
GET /model-info            name, version, type, thresholds
GET /subscribers/sample    valid identifiers for the UI
GET /subscriber/{msno}/features read-only feature vector
POST /predict              predict from supplied feature values
POST /predict/{msno}       PRIMARY: predict by member identifier
  
```

Table 4.2 API Endpoint Specification

Method	Path	Purpose	Error Responses
GET	/health	Service status, model availability and version	—
GET	/model-info	Registered model metadata, feature count, risk thresholds	—
GET	/subscribers/sample	A small set of valid member identifiers	503 if the feature table is unavailable
GET	/subscriber/{msno}/features	Read-only engineered feature vector	404 unknown identifier; 422 malformed
POST	/predict	Prediction from supplied feature values; omitted features are filled from training medians and modes	422 malformed or out-of-range; 503 model unavailable
POST	/predict/{msno}	Primary user-facing prediction by member identifier	404 unknown identifier; 422 malformed; 503 model unavailable

4.11 Member-ID Prediction Flow

The primary prediction path accepts only a member identifier. The service validates the identifier, confirms its presence in the feature table, retrieves that subscriber's engineered feature vector, normalises the categorical representation to match the fitted encoder, obtains a probability from the registered model, and derives the predicted label and risk level from configurable thresholds.

Retrieving features server-side rather than accepting them from the client is a design decision rather than a convenience. It ensures that the feature definition used at inference is exactly the one produced by the feature engineering layer, eliminating the possibility of the two diverging.

Figure 4.7 Member-ID Prediction Flow

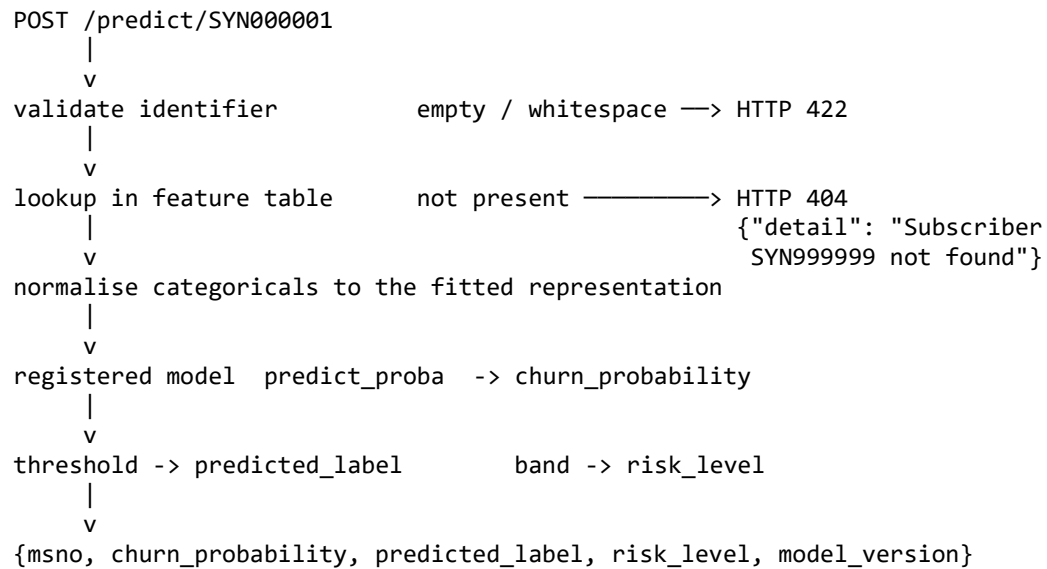


Table 4.3 Risk Level Thresholds

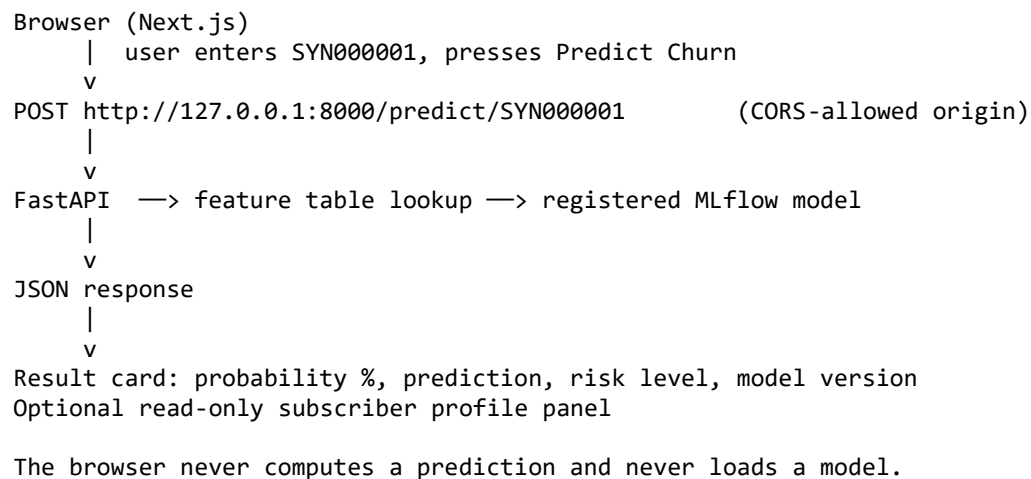
Churn probability	Risk level	Configuration key
0.00 to 0.39	LOW	CHURN_RISK_LOW_THRESHOLD = 0.40
0.40 to 0.69	MEDIUM	between the two thresholds
0.70 to 1.00	HIGH	CHURN_RISK_HIGH_THRESHOLD = 0.70

Thresholds are defined once in the central configuration module and exposed through the model information endpoint, so the interface displays the bands actually in force rather than a duplicated copy of them.

4.12 Frontend Architecture

The web interface presents a single input accepting a member identifier. It performs no feature engineering and holds no model; on submission it issues a request to the prediction endpoint and renders the response. Service status shown in the interface is obtained from the health and model information endpoints rather than hard-coded.

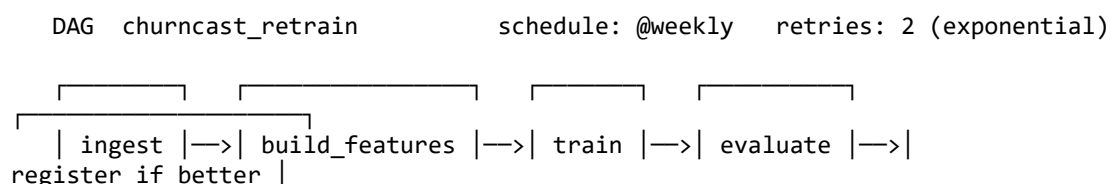
Figure 4.8 Frontend to API to Model Prediction Flow

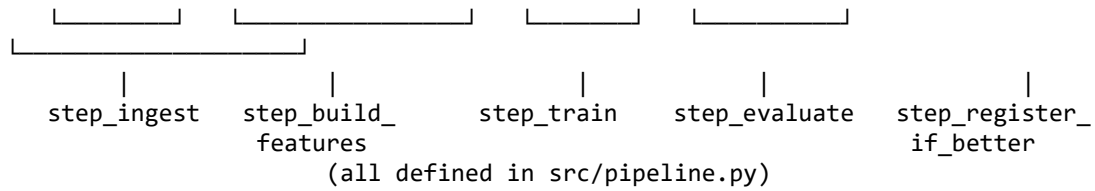


4.13 Airflow Architecture

Retraining is expressed as a directed acyclic graph named `churncast_retrain`, scheduled weekly, comprising five tasks in linear dependency. Each task is a thin wrapper over a function in the shared pipeline module, so the orchestration layer carries scheduling concerns only and no business logic is duplicated. Tasks are configured with two retries and exponential backoff.

Figure 4.9 Airflow Retraining DAG





```
register_if_better: candidate val ROC-AUC > incumbent ? register : retain
```

Implementation status: The DAG is implemented and structurally validated; scheduler-level runtime execution remains a deployment verification step.

4.14 Evidently Monitoring Architecture

Monitoring compares a reference dataset against a current dataset. The reference is the training cohorts' feature distribution; the current dataset is a later cohort. Eight features spanning recency, frequency, engagement, subscription and lifecycle are monitored, and the report records a per-feature divergence score together with the count of features flagged as drifted.

A controlled shift may optionally be injected into the current dataset to demonstrate that the monitoring detects change. That mode is labelled explicitly in the output as a synthetic drift demonstration and does not represent observed production drift.

Figure 4.10 Evidently Drift Monitoring Flow

```

feature table
├── reference = cohorts 2024-07 .. 2024-10    n = 5,030
├── current   = cohort  2024-12              n = 1,235
│
│ [optional] inject_synthetic_drift()  <-- demonstration mode only
│
Dataset.from_pandas(..., DataDefinition(numerical, categorical))
│
Report(metrics=[DataDriftPreset()]).run(current, reference)
│
├── reports/drift_report_<date>.html
├── reports/drift_summary_<mode>.json    drifted count + per-column score

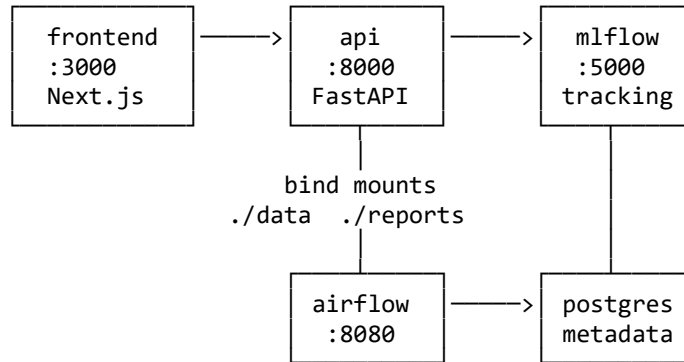
```

4.15 Docker Architecture

Containerisation is specified as a Compose definition coordinating five services. The application image is built on Python 3.11 rather than the host interpreter because Apache Airflow has no release compatible with Python 3.13, and using one interpreter across the

API, the pipeline and the orchestrator keeps the executed code path identical between services.

Figure 4.11 Docker Compose Architecture



Shared volumes let a pipeline run in either api or airflow be visible to the other and on the host.

Implementation status: *The Docker configuration is configured and prepared for containerized execution; runtime verification is pending Docker Engine availability.*

4.16 Storage and Artifact Design

The system uses file-based storage throughout, which is appropriate for a prototype and keeps every intermediate artifact inspectable. Raw tables are comma-separated files; the engineered feature table is persisted as a single file that is both the training input and the serving lookup source, which is what guarantees that the two see identical values. MLflow uses an embedded SQLite backend for run metadata and a local directory for model artifacts. Evaluation figures, metric documents and drift reports are written to the reports directory, from which the figures reproduced in Chapter 6 are taken directly.

CHAPTER 5

IMPLEMENTATION

5.1 Development Environment

Table 5.1 Development Environment

Aspect	Detail
Operating system	Windows 11
Interpreter	Python 3.13.5 in an isolated virtual environment
Dependency management	Pinned requirements file
Version control	Git; repository hosted on GitHub
Code quality	Ruff for linting and Black for formatting, via pre-commit hooks; Ruff reports no outstanding findings across src, scripts, tests and pipelines
Frontend toolchain	Node.js 22.17.0 with npm 10.9.2
Determinism	A single seed constant (42) governs dataset generation, partitioning and model randomness

5.2 Dataset Generation and Data Preparation

5.2.1 Rationale for a Synthetic Prototype Dataset

The original KKBOX extracts proved impractical to use as the basis for a complete, reproducible pipeline within the project period, for reasons documented in Section 5.2.3. A deterministic synthetic dataset mirroring the structure and semantics of the KKBOX tables was therefore constructed so that the entire MLOps workflow could be developed, executed and verified end to end. Because every downstream module depends on the column contract rather than on the generator, the original extracts can replace the synthetic data through a configuration change.

5.2.2 Composition of the Synthetic Dataset

The generator assigns each subscriber an observation cutoff in one of six monthly cohorts and writes the renewal transaction that determines the label strictly after that cutoff. Churn probability is a logistic function of latent subscriber traits comprising engagement level, engagement trend, automatic renewal propensity, plan price, tenure and cancellation history,

with additive noise so that the resulting classification problem is learnable but not trivially separable.

Table 5.2 Synthetic Prototype Dataset Composition

File	Rows	Columns	Contents
members.csv	7,500	6	City, age, gender, registration channel and date
transactions.csv	69,590	9	Plan days, list price, amount paid, auto-renew flag, cancel flag, transaction and expiry dates
user_logs.csv	519,012	9	Daily play counts by completion bucket, unique tracks, total seconds
train.csv	7,500	2	Member identifier and binary churn label
cohorts.csv	7,500	3	Member identifier, cohort month, observation cutoff

The observed churn rate is 0.3036. Member identifiers are deterministic, running from SYN000001 to SYN007500, which allows the member-identifier prediction flow to be demonstrated reliably.

5.2.3 Relationship to the Original KKBOX Data

The original KKBOX extracts are preserved in a separate directory and are not read by the prototype pipeline. Their contents were verified directly during preparation of this report, and the verification produced a finding that must be stated explicitly.

Table 5.3 Preserved Original KKBOX Extracts

File	Size	Verified content	Provenance
train.csv	45.6 MB	970,960 labelled subscribers	Original KKBOX extract
members.csv	427.9 MB	Subscriber demographic records	Original KKBOX extract
transactions.csv	115.4 MB	1,197,050 unique subscribers	Original KKBOX extract (renewal-window variant)
WSDMChurnLabeller.scala	7.1 KB	Official label generation logic	Supplied with the competition data
user_logs.csv	96.3 MB	20,000 subscribers, covering 1.61% of the labelled population, spanning 2016-11-02 to 2017-02-28	NOT original: generated by a since-deprecated helper script

The final row records a data integrity issue identified during this work. The listening-log file held alongside the original extracts is not the genuine KKBOX listening data; it was produced by a helper script that overwrote the original file and covers only 1.61 percent of the labelled subscriber population. The genuine KKBOX listening extract was therefore

never available to this project. The script responsible has been disabled and now terminates immediately with an explanatory message, so the situation cannot recur.

This finding reinforces the separation maintained throughout this report. All experimental results in Chapter 6 correspond to the reproducible synthetic KKBOX-like prototype dataset. No evaluation on the official KKBOX benchmark has been performed, and none is claimed.

5.3 Feature Engineering

The feature engineering module implements the leakage rule described in Section 4.5. Each source table is merged with the cohort table, filtered so that only records dated on or before the subscriber's own cutoff survive, and then reduced by group-wise aggregation. Transaction features are computed after sorting each subscriber's history by date, so that the most recent transaction is well defined rather than dependent on row order.

Table 5.4 Engineered Feature Groups

Group	Features	Source
Recency	recency_days, days_since_last_listen	transactions, user_logs
Transaction frequency	txns_last_30d, txns_last_90d, transaction_frequency, prior_cancellations	transactions
Listening frequency and volume	active_days_last_30d, active_days_last_90d, total_secs_last_30d, total_secs_last_90d, unique_songs_last_30d, unique_songs_last_90d, avg_daily_secs	user_logs
Behavioural quality	skip_ratio, completion_ratio, engagement_trend, recent_activity_ratio	user_logs
Subscription economics	plan_price, payment_amount, payment_consistency, is_auto_renew, payment_method_last	transactions
Lifecycle and demographic	tenure_days, age, gender, city, registered_via	members

This yields 27 features, of which 22 are numerical and 5 categorical. The engagement trend feature encodes direction rather than level: listening records in the four weeks preceding the cutoff are assigned to weekly buckets, the mean listening seconds is computed per bucket, and a least-squares slope is fitted across buckets ordered forward in time, so that a negative value indicates declining engagement approaching the cutoff.

Subscribers lacking history in a given source are retained rather than dropped. Count features are filled with zero, while recency and ratio features remain missing, which preserves the

distinction between an observed zero and an unknown quantity. Binary indicators record the presence of each history type.

5.4 Model Development

Two candidates are trained within a single invocation on an identical partition so that their comparison is not confounded by differing data.

Table 5.5 Model Hyperparameters

Model	Configuration
Logistic Regression	max_iter = 2000; class_weight = balanced; random_state = 42; wrapped in CalibratedClassifierCV with sigmoid calibration over 3 folds
Gradient Boosting (XGBoost)	n_estimators = 400; max_depth = 4; learning_rate = 0.05; subsample = 0.85; colsample_bytree = 0.85; reg_lambda = 1.2; eval_metric = auc; scale_pos_weight set to the negative-to-positive ratio of the training partition; random_state = 42; wrapped in the same calibration stage

Class imbalance is addressed through class weighting within each learning algorithm rather than by synthetic resampling. Interpolating synthetic minority examples would create records belonging to no cohort, which would undermine the temporal guarantee that the split is designed to provide. Since the prototype churn rate of 0.3036 is moderate rather than extreme, weighting is sufficient.

Calibration is applied because the predicted value is used operationally as a risk level and is mapped to risk bands, which requires a probability rather than a ranking score. Sigmoid calibration was selected over isotonic regression given the size of the validation cohort, isotonic regression being more prone to overfitting on smaller samples.

5.5 Model Evaluation

Model selection uses validation ROC-AUC alone. The selected model is then scored exactly once on the test cohort, and that figure is never used to choose between candidates, tune a threshold or decide promotion. Six metrics are computed at a decision threshold of 0.5: ROC-AUC, PR-AUC, accuracy, precision, recall and F1-score. The confusion matrix, ROC curve, precision-recall curve and per-class classification report are written to the reports directory and attached to the winning MLflow run.

5.6 MLflow Tracking

Tracking is integrated into the training module rather than implemented separately, so that no run can occur without being recorded. The experiment is named ChurnCast and the tracking backend is an embedded SQLite database.

```
with mlflow.start_run(run_name=name) as run:
    mlflow.set_tags({"model_type": name, "dataset": "synthetic-kkbox-like",
                    "split": "temporal"})
    mlflow.log_params(params)
    mlflow.log_params({"seed": config.SEED,
                       "feature_count": len(FEATURE_COLUMNS),
                       "n_train": len(X_train), "n_val": len(X_val),
                       "n_test": n_test, "calibrated": True,
                       "calibration_method": "sigmoid"})
    ...
    mlflow.log_metrics({f"val_{k}": v for k, v in val_metrics.items()})
    mlflow.sklearn.log_model(model, name="model",
                             input_example=X_train.head(3),
                             serialization_format="cloudpickle")
```

5.7 Model Registry

Registration is conditional. The promotion gate resolves each already-registered version back to its originating run, reads the validation ROC-AUC recorded there, and registers the candidate only if it strictly improves on the best incumbent value. The decision, whether acceptance or rejection, is written into the metrics document so that the registry state is explainable from the artifacts alone.

```
incumbent = registered_best_roc_auc(client, model_name)
candidate = result["val_metrics"]["roc_auc"]

if incumbent is not None and candidate <= incumbent:
    return None, (f"candidate val_roc_auc={candidate:.4f} did not beat "
                  f"incumbent {incumbent:.4f}; keeping existing model")

mv = mlflow.register_model(f"runs://{result['run_id']}/model", model_name)
```

5.8 FastAPI Implementation

Request and response schemas are declared as Pydantic models, so malformed payloads are rejected before reaching the model and the OpenAPI specification is generated automatically. Numeric fields carry range constraints, so physically impossible values such as a negative recency or a ratio above one are rejected rather than scored. Cross-origin access is restricted to configured origins.

5.9 Member-ID Prediction

The member-identifier endpoint validates the identifier, looks the subscriber up in the persisted feature table, coerces categorical columns to the representation the encoder was fitted on, and scores the row with the registered model. An unknown identifier produces a 404 response carrying an explanatory message; an empty or whitespace-only identifier produces a 422.

5.10 Next.js Frontend

The interface is a single page built with Next.js. It presents one input for the member identifier, sample identifiers retrieved from the service, and a result card displaying the churn probability as a percentage, the prediction in plain language, the risk level as a coloured band and the model version. A collapsible panel shows the subscriber's read-only feature values, retrieved from the feature endpoint.

The client performs no computation on the model output beyond formatting. Loading state disables the submit control to prevent duplicate requests, and the error states distinguish an unknown subscriber, an invalid identifier and an unavailable service, each rendered as a readable message rather than a stack trace.

5.11 Airflow DAG

The DAG defines five tasks in linear dependency, each delegating to the shared pipeline module. Default arguments specify two retries with exponential backoff and a bounded maximum delay. The task callables return compact summaries rather than fitted estimators, keeping the inter-task payload small.

```
with DAG(dag_id="churncast_retrain",
        default_args={"retries": 2,
                      "retry_delay": timedelta(minutes=2),
                      "retry_exponential_backoff": True,
                      "max_retry_delay": timedelta(minutes=15)},
        schedule="@weekly", catchup=False, max_active_runs=1) as dag:

    ingest >> build_features >> train >> evaluate >> register_if_better
```

Implementation status: *The DAG is implemented and structurally validated; scheduler-level runtime execution remains a deployment verification step. Its structure was validated by parsing*

the module and confirming the task identifiers and their order, and the pipeline functions it invokes are the same ones exercised by the pipeline script and the test suite.

5.12 Evidently Drift Monitoring

Monitoring is implemented against the Evidently 0.7 interface, which differs substantially from earlier releases. The reference and current frames are wrapped as datasets with an explicit data definition separating numerical from categorical columns, and a report constructed from the data drift preset is executed and written to HTML. A summary document records the drifted-feature count and the per-column divergence score.

5.13 Docker Configuration

Two images are defined. The application image installs the pinned requirements and copies the source, scripts and pipeline definitions, exposing the API port and declaring a health check against the health endpoint. The frontend image builds the Next.js application and serves it. The Compose file coordinates five services with declared dependencies and health conditions, bind-mounting the data and reports directories so that artifacts are shared between services and visible on the host.

***Implementation status:** The Docker configuration is configured and prepared for containerized execution; runtime verification is pending Docker Engine availability.*

5.14 Testing

The test suite comprises 74 tests organised by concern. Synthetic fixtures with hand-computed expected values are preferred to samples of the generated data, because a fixture with a known answer tests the implementation rather than recording its current behaviour.

Table 5.6 Test Suite Composition

Module	Focus
test_data.py	Generator determinism under a fixed seed; required columns; referential integrity of member identifiers; identifier format; label domain; plausible churn rate; cohort span; confirmation that renewal transactions fall after the cutoff
test_features.py	Presence of every declared feature; bounded ratios; non-negative counts; 30-day windows never exceeding their 90-day counterparts; exclusion of forbidden source columns; string-typed categoricals; leakage guards; assertion that no single feature separates the label
test_model.py	Temporal split disjointness, coverage and chronological ordering; training and probability bounds; metric computation; consistency of the persisted metrics document; train/serve categorical agreement; agreement between the registered model and the reported test metrics

test_api.py	Health and model information; sample identifiers; feature retrieval; both prediction paths; risk band and label consistency; 404 and 422 handling; prediction determinism; OpenAPI completeness
test_monitoring.py	Reference and current partitioning; injected drift changing the distribution; HTML report generation; drift detection; MLflow run and registry state; rejection of a worse candidate by the promotion gate

The leakage tests are the most important in the suite. Each constructs a fixture containing records on both sides of the cutoff, computes the features, then adds a deliberately extreme post-cutoff record and recomputes, asserting that the two results are identical.

```
def test_post_cutoff_transactions_are_ignored():
    cohorts, tx, logs, cutoff = _fixture_frames()
    before = _transaction_features(tx, cohorts)

    leaked = pd.concat([tx, pd.DataFrame([{"msno": "A", "plan_list_price": 9999, "is_cancel": 1,
        "transaction_date": cutoff + pd.Timedelta(days=5), ...}])])
    after = _transaction_features(leaked, cohorts)

    pd.testing.assert_frame_equal(before, after)
```

5.15 Debugging and Engineering Fixes

Four defects encountered during development are recorded here because each illustrates a class of fault that produces no error and is therefore invisible to ordinary testing.

Table 5.7 Engineering Defects Identified and Resolved

No.	Defect	Detection	Resolution
1	Target leakage: a single global observation cutoff fell inside the label period, so transaction features encoded whether the renewal had occurred	Conditional churn rates showed a step discontinuity at the label-window boundary rather than a gradient; a fixed-population window-retraction experiment showed the score declining monotonically as access to the label period was withdrawn	Replaced with a per-subscriber cutoff derived from the label definition; leakage guards added to the test suite
2	Train/serve skew: the automatic-renewal flag was represented as the string "1" during training but read back as the float 1.0 after a round trip through the persisted feature table	A test asserting that categorical features are string-typed after loading	A single normalisation function applied on both write and read, so the two paths produce identical values
3	Train/serve skew, second occurrence: the training module read the persisted feature table with a plain CSV read, returning city and registration channel as integers, while the service loaded the same file through the normalising loader and obtained strings; the encoder therefore silently discarded both columns at inference	A verification comparing the registered model's score on the held-out cohort against the value recorded in the metrics document revealed a discrepancy of 0.0038 ROC-AUC	The training module now uses the same loader as the service; two regression tests were added, one asserting representational agreement and one asserting that the registered model reproduces the reported metrics exactly

4	Serialisation and monitoring interface faults: the tracking library's default serialisation format rejected the calibration wrapper's internals, and the monitoring library interpreted the member identifier index as a time axis	Both surfaced as exceptions during the first execution of the affected stage	Cloudpickle serialisation was selected explicitly; the frames passed to the monitoring layer are given a reset index
---	--	--	--

The third defect merits emphasis because of how it was found. The model was performing correctly during training and the API was returning plausible probabilities; nothing failed. The discrepancy became visible only when the registered model was reloaded and scored against the same held-out cohort that the metrics document described, and the two disagreed. Every metric reported in Chapter 6 has since been verified to be reproducible from the registered model, and that verification is now an automated test rather than a manual check.

CHAPTER 6

RESULTS AND DISCUSSION

Every figure in this chapter was produced by executing the implementation described in Chapter 5, and every number is traceable to an artifact in the reports directory. The results correspond to the reproducible synthetic KKBOX-like prototype dataset described in Section 5.2 and are not results on the official KKBOX benchmark.

6.1 Dataset Summary

Table 6.1 Dataset Summary

Property	Value
Dataset	Synthetic KKBOX-like prototype
Subscribers	7,500
Transaction records	69,590
Listening records	519,012
Churn rate	0.3036
Random seed	42
Cohort months	2024-07 to 2024-12 (six cohorts)
Training partition	5,030 subscribers (2024-07 to 2024-10)
Validation partition	1,235 subscribers (2024-11)
Test partition	1,235 subscribers (2024-12)

6.2 Feature Summary

The feature engineering layer produces 27 features per subscriber, comprising 22 numerical and 5 categorical attributes across the six groups listed in Table 5.4. Every labelled subscriber is retained: the feature table contains exactly 7,500 rows, matching the labelled population, with explicit indicators marking the absence of transaction or listening history rather than excluding those subscribers.

Diagnostics emitted during the build report one feature exceeding the missingness threshold. The age attribute is approximately 35 percent missing, which is a deliberate property of the generator: it emits a zero for unknown ages, mirroring the corresponding convention in the original KKBOX member records, and the feature layer converts out-of-range ages to missing values for median imputation. No feature is constant.

6.3 Model Performance

Both candidates were trained on an identical partition within a single invocation. Model selection used validation ROC-AUC alone.

Table 6.2 Validation Performance of Candidate Models

Model	ROC-AUC	PR-AUC	Accuracy	Precision	Recall	F1
Logistic Regression	0.7835	0.6966	0.7733	0.7049	0.4526	0.5513
Gradient Boosting (XGBoost)	0.7805	0.7028	0.7757	0.6929	0.4868	0.5719

Logistic Regression was selected, achieving the higher validation ROC-AUC by a margin of 0.0030. The two models are close, and the comparison is instructive: the gradient boosted ensemble attains higher PR-AUC, accuracy and recall, while the linear model attains higher ROC-AUC and precision. Since the selection criterion was declared in advance as validation ROC-AUC, the linear model was chosen, and that decision was not revisited after the test score became available.

That the additional capacity of a boosted ensemble yields no clear advantage here is consistent with the structure of the generator, in which churn probability is a logistic function of the latent traits. A linear model in the correctly engineered feature space is therefore close to the form of the underlying process. This is a property of the synthetic data and should not be generalised to the original KKBOX data, where the relationship between behaviour and churn is unlikely to be so nearly linear.

Table 6.3 Held-Out Test Performance of the Selected Model

Model	Partition	ROC-AUC	PR-AUC	Accuracy	Precision	Recall	F1
Logistic Regression	Test (2024-12)	0.7957	0.6904	0.7951	0.7309	0.4579	0.5630

The test cohort was scored exactly once, after selection was complete. Test ROC-AUC of 0.7957 slightly exceeds the validation figure of 0.7835, which indicates that the selection did not overfit the validation cohort.

6.4 ROC-AUC

ROC-AUC is the primary metric because it is invariant to class prior and therefore comparable across cohorts whose churn rates differ. The selected model attains 0.7835 on validation and 0.7957 on the held-out test cohort. A value of 0.5 would indicate no discriminative ability; the observed values indicate that the model orders subscribers by risk substantially better than chance while remaining well short of separability, which is the expected regime for a churn problem with genuine irreducible noise.

6.5 PR-AUC

Area under the precision-recall curve is reported because it is more informative than ROC-AUC under class imbalance, being sensitive to performance on the minority class specifically. The selected model attains 0.6966 on validation and 0.6904 on test. Against a base churn rate of 0.3036, which is the PR-AUC a random classifier would achieve, this represents a substantial improvement in the precision attainable at a given level of recall.

6.6 Accuracy

Accuracy on the test cohort is 0.7951. This metric is reported for completeness but is not used for selection. Because 71.2 percent of the test cohort did not churn, a classifier predicting the negative class unconditionally would attain 0.7117 accuracy while being operationally worthless; the observed improvement over that baseline is therefore a weaker signal than the ranking metrics.

6.7 Precision

Precision on the test cohort is 0.7309. Of the subscribers the model flagged as likely to churn at the default threshold, approximately 73 percent did churn. For a retention team this is the directly actionable quantity, since it determines the proportion of outreach effort directed at subscribers who genuinely required it.

6.8 Recall

Recall on the test cohort is 0.4579, meaning the model identifies approximately 46 percent of subscribers who actually churned at the default threshold of 0.5. This is the weakest aspect of

the result and is discussed in Section 6.20. The threshold is a configurable parameter rather than a property of the model: lowering it would increase recall at the cost of precision, and the appropriate operating point depends on the relative cost of a missed churner against a wasted retention contact.

6.9 F1 Score

The F1 score on the test cohort is 0.5630, reflecting the imbalance between precision and recall noted above. The gradient boosted candidate attained a higher validation F1 of 0.5719, a consequence of its higher recall, which illustrates that the choice of selection metric materially affects which model is chosen.

6.10 Confusion Matrix

Table 6.4 Per-Class Classification Report on the Test Cohort

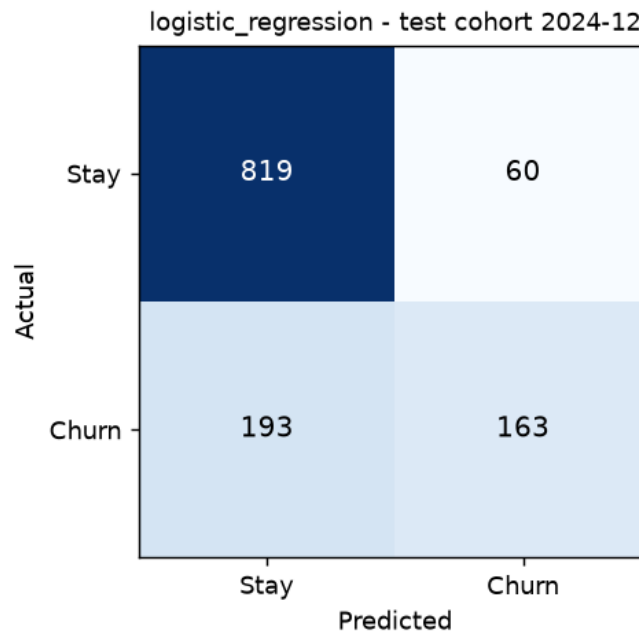
Class	Precision	Recall	F1-score	Support
Stay (0)	0.81	0.93	0.87	879
Churn (1)	0.73	0.46	0.56	356
Accuracy			0.80	1,235
Macro average	0.77	0.69	0.71	1,235
Weighted average	0.79	0.80	0.78	1,235

Table 6.5 Confusion Matrix Values

	Predicted: Stay	Predicted: Churn
Actual: Stay	819 (true negative)	60 (false positive)
Actual: Churn	193 (false negative)	163 (true positive)

The matrix shows the model to be conservative at the default threshold. It produces few false positives, 60 against 819 correctly retained subscribers, which is why precision is high. It misses 193 of 356 actual churners, which is why recall is moderate. For a retention application this bias is defensible, since a false positive consumes a contact whereas a false negative loses a subscriber, but the threshold should be tuned to the actual cost ratio before operational use.

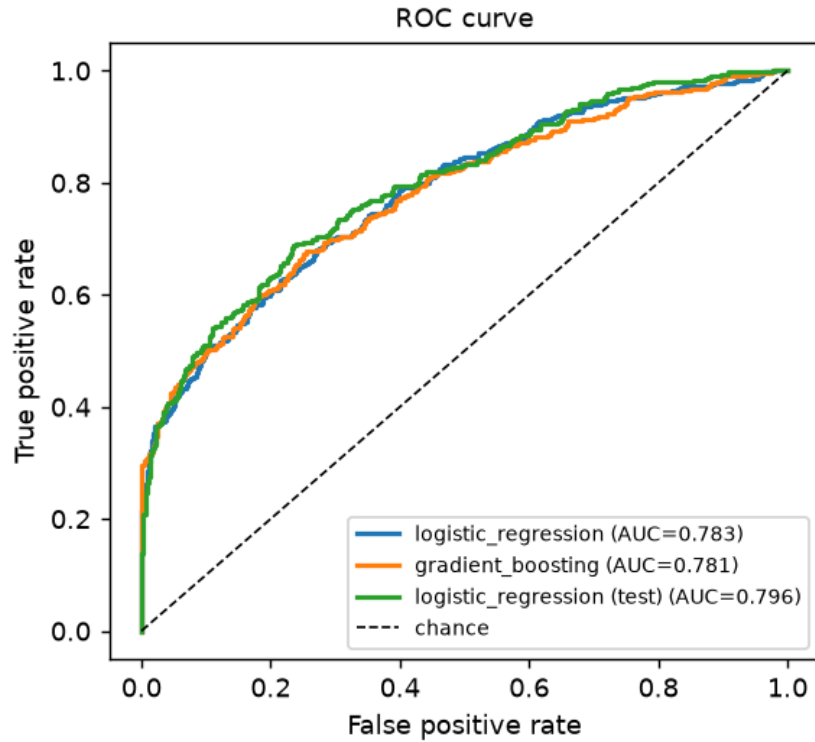
Figure 6.1 Confusion Matrix on the Held-Out Test Cohort



6.11 ROC Curve

The receiver operating characteristic curves for both candidate models on the validation cohort, together with the selected model on the test cohort, are shown below. The proximity of the two candidate curves reflects the narrow margin recorded in Table 6.2.

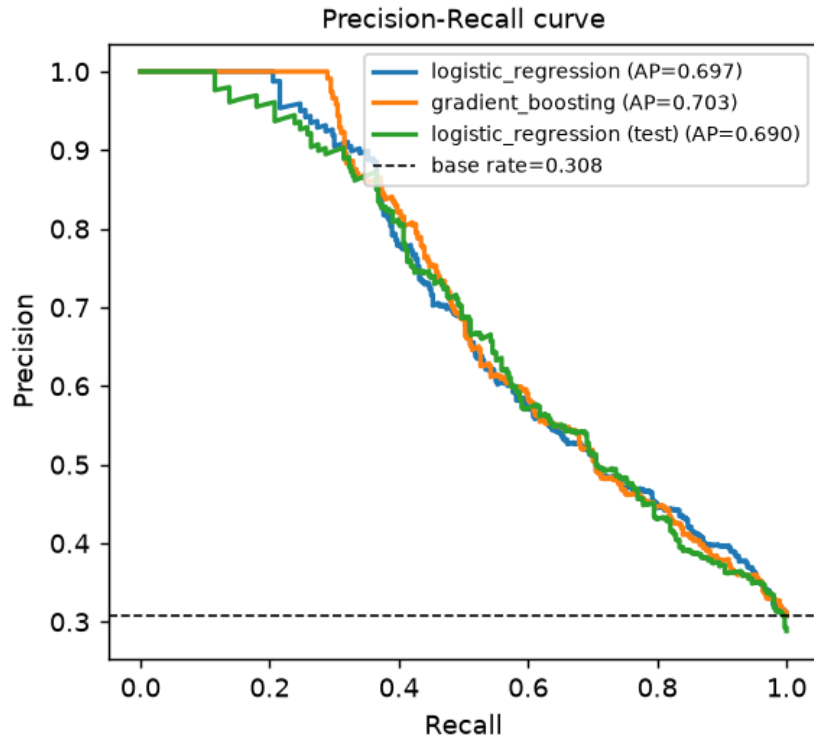
Figure 6.2 ROC Curves for Validation and Test Evaluation



6.12 Precision-Recall Curve

The precision-recall curves are shown against the base rate of the cohort, which is the level a random classifier would attain. The curves remain well above that line across the recall range, confirming that the ranking carries information about the minority class rather than merely reflecting the class prior.

Figure 6.3 Precision-Recall Curves for Validation and Test Evaluation



6.13 MLflow Results

The training session recorded one tracked run per candidate under the ChurnCast experiment. Each run captured its hyperparameters, the seed, the feature count, the three partition sizes, the calibration configuration and six validation metrics; the winning run additionally carries the six test metrics and the evaluation artifacts.

Table 6.6 MLflow Tracked Runs and Registry State

Property	Value
Experiment	ChurnCast
Tracked runs in the reported session	2 (one per candidate model)
Winning run identifier	542b99cd60ff4fceb2d6260392048640
Logged artifacts	confusion_matrix.png, roc_curve.png, precision_recall_curve.png, classification_report.txt, model_comparison.csv, model_metrics.json
Registered model name	ChurnCastModel
Registered version	1
Registry decision	registered ChurnCastModel v1 (val_roc_auc=0.7835, first version)
Serialisation format	cloudpickle

6.14 Model Registry Results

The promotion gate was observed exercising both of its branches during development. In an earlier session a candidate improving on the incumbent was accepted, and in a subsequent session a candidate matching the incumbent exactly was rejected:

```
registered ChurnCastModel v2 (val_roc_auc=0.7835 > incumbent 0.7814)

candidate val_roc_auc=0.7835 did not beat incumbent 0.7835;
keeping existing model
```

The second case is the more important of the two. A gate that only ever accepts is indistinguishable from no gate; observing a rejection confirms that the comparison is genuinely performed. The rejection of an exactly tied candidate also reflects a deliberate choice of a strict inequality, so that repeated identical runs do not accumulate redundant registry versions.

In the final clean-slate session reported here the registry was empty at the outset, so the candidate was registered as the first version without a comparison being possible, and the decision string records that fact.

6.15 Member-ID Prediction Results

The member-identifier endpoint was exercised against the registered model for five subscribers spanning the risk range, together with an identifier absent from the dataset.

Table 6.7 Member-ID Prediction Results

Member ID	Churn probability	Predicted label	Risk level	Model version
SYN000001	0.3060	0 (likely to stay)	LOW	1
SYN000002	0.4319	0 (likely to stay)	MEDIUM	1
SYN000003	0.1149	0 (likely to stay)	LOW	1
SYN000004	0.6428	1 (likely to churn)	MEDIUM	1
SYN000005	0.0857	0 (likely to stay)	LOW	1
SYN999999	—	—	—	HTTP 404: Subscriber SYN999999 not found

The five subscribers receive distinct probabilities spanning two risk bands and both predicted labels, which confirms that the service is scoring each subscriber's own feature vector rather than returning a constant. Repeated requests for the same identifier return identical values, verified by an automated determinism test.

The differences are explicable from the underlying feature values. The subscriber assigned the highest probability has automatic renewal disabled and a comparatively short tenure, whereas the subscriber assigned the lowest has an extended tenure and active renewal, which are the directions the generator's churn model encodes.

```
POST /predict/SYN000001

{
  "msno": "SYN000001",
  "churn_probability": 0.306,
  "predicted_label": 0,
  "risk_level": "LOW",
  "model_version": "1"
}
```

6.16 Frontend Results

The web interface was exercised in a browser against the running service. Entering a valid member identifier and submitting produced the prediction returned by the service, rendered as a percentage together with the plain-language prediction, the risk band and the model version. The status indicators in the interface footer displayed the model version, model type and subscriber count obtained from the health and model information endpoints.

The collapsible subscriber profile displayed feature values matching those returned by the feature endpoint for the same identifier, confirming that the panel reports stored values rather than recomputing them. Three interaction paths were verified: a successful prediction, an unknown identifier producing a readable "Subscriber not found" message rather than an error page, and a whitespace-only entry producing a validation message.

6.17 Drift Monitoring Results

Drift monitoring was executed in both modes over the eight monitored features, with the training cohorts as reference and the latest cohort as current.

Table 6.8 Data Drift Monitoring Results

Mode	Reference	Current	Features drifted	Interpretation
Natural	5,030 rows (2024-07 to 2024-10)	1,235 rows (2024-12)	0 of 8	Cohorts drawn from the same generative process show no distributional change, which is the expected result and confirms

				the monitor does not report spurious drift
Injected demonstration	5,030 rows (2024-07 to 2024-10)	1,235 rows (2024-12), deliberately shifted	6 of 8 (share 0.75)	A controlled shift in recency, activity, listening volume, price and renewal status is detected, confirming the monitor responds to genuine distributional change

The two features not flagged under the injected shift are transaction count and tenure, neither of which the injection modifies; their divergence scores are unchanged between the two modes, at 0.0135 and 0.0284 respectively. This provides a useful internal control: the monitor flags exactly the features that were perturbed and leaves the others alone.

The injected mode is a synthetic drift demonstration. The shift is applied deliberately by the monitoring module to exercise the detection path, and the generated summary carries an explicit note to that effect. It does not represent observed production drift, and no production deployment exists from which such drift could have been observed.

6.18 Testing Results

Table 6.9 Test Execution Summary

Property	Value
Test framework	pytest 9.1.1
Total tests	74
Passed	74
Failed	0
Execution time	approximately 20 seconds
Linting	Ruff reports no findings across src, scripts, tests and pipelines

Two tests are singled out because they encode the project's central concerns. The leakage guards add a deliberately extreme post-cutoff record to a fixture and assert that every computed feature value is unchanged. The registered-model agreement test reloads the registered model, scores the held-out cohort and asserts that the resulting ROC-AUC matches the value recorded in the metrics document to within 1e-9; this test was added after the defect described in row 3 of Table 5.7 and would have detected it immediately.

6.19 Pipeline Reproducibility

Reproducibility was verified by deleting the tracking database, the artifact store and the persisted feature table, then re-executing the complete pipeline from the dataset generation step with the force flag. The regenerated dataset, the partition sizes and the reported metrics were identical to the previous execution, and the member-identifier predictions returned by the service were unchanged. A determinism test asserts that the generator produces identical output for a given seed.

The complete pipeline, from dataset generation through feature engineering, training of both candidates, evaluation, registration and drift reporting, completes in approximately 60 seconds on the development machine.

6.20 Discussion and Limitations

6.20.1 Interpretation of the Results

The reported test ROC-AUC of 0.7957 describes a model that ranks subscribers by churn risk substantially better than chance under an evaluation protocol designed to resist optimistic bias. The protocol matters more than the figure: the cutoff is per-subscriber and derived from the label definition, the evaluation cohort is later in time than the cohorts used for fitting and selection, the held-out cohort was scored once, and the absence of leakage is asserted by tests rather than assumed.

It is worth stating plainly why no result near 0.90 is reported. An earlier iteration of this project did produce such a figure, and the diagnostic work described in Section 1.3 established that it was substantially attributable to target leakage rather than to predictive ability. That figure is not a result of this project and is not presented as one. The lower figure reported here is the more valuable of the two, because it describes something the system can actually do.

6.20.2 Limitations of the Data

The results correspond to synthetic data whose churn mechanism is known by construction. This is a genuine limitation: the generator encodes a logistic relationship between latent traits and churn, so the problem is more regular than real subscriber behaviour, and the near-

equivalence of the linear and boosted models observed in Section 6.3 is likely an artefact of that regularity. The figures should be read as evidence that the pipeline is correct and reproducible, not as an estimate of performance on real subscriber data.

No evaluation on the official KKBOX benchmark has been performed. The preserved original extracts are incomplete for this purpose, as recorded in Section 5.2.3: the listening-log file retained alongside them is not the genuine KKBOX listening data, and the transaction extract available is the renewal-window variant rather than the full subscription history.

6.20.3 Limitations of the Model

Recall of 0.4579 at the default threshold means the model misses slightly more than half of the subscribers who go on to churn. The decision threshold has not been tuned against a business cost ratio, and no operating point analysis is presented. No feature attribution is produced per prediction, so the service returns a score without an explanation of what drove it.

6.20.4 Components Not Verified at Runtime

Two components are implemented but were not verified in execution, and this is stated here rather than left implicit.

- **Airflow orchestration.** The DAG is implemented and structurally validated; its task identifiers, dependency order and callables were confirmed by parsing the module, and the pipeline functions it invokes are the same ones exercised by the pipeline script and the test suite. Scheduler-level runtime execution remains a deployment verification step. No Apache Airflow release supports the Python 3.13 interpreter used on the development host, which is why the DAG targets the container environment.
- **Docker deployment.** The Dockerfile, frontend Dockerfile, Compose definition and ignore files are implemented, and the Compose definition parses and declares five services with the intended ports and dependencies. Runtime verification is pending Docker Engine availability. Docker Desktop 4.91.0 was subsequently installed on the development machine and its command-line client verified as functional, but the engine did not complete first-run provisioning: the required backend distribution was never

created and the daemon did not become available. No image was therefore built and no container was started.

Accordingly, this report does not claim that the retraining DAG executed or that the containerised stack was deployed. Both are recorded in Table 6.10 with their actual status.

Table 6.10 Component Implementation and Verification Status

Component	Status	Evidence
Synthetic dataset generation	Verified	Dataset regenerated from a clean state with identical output under seed 42
Data validation	Verified	Coverage and class-balance diagnostics emitted on every build
Leakage-aware feature engineering	Verified	Post-cutoff guard tests pass; 27 features produced for all 7,500 subscribers
Temporal split	Verified	Cohort ordering, disjointness and coverage asserted by tests
Logistic Regression	Verified	Validation and test metrics recorded in model_metrics.json
Gradient Boosting	Verified	Validation metrics recorded in model_metrics.json
Probability calibration	Verified	Applied to both candidates; configuration logged with each run
MLflow tracking	Verified	Two tracked runs with parameters, metrics, tags and artifacts
Model registry and promotion gate	Verified	ChurnCastModel v1 registered; gate observed both accepting and rejecting candidates
FastAPI service	Verified	All six endpoints exercised; error paths confirmed
Member-ID prediction	Verified	Five subscribers scored with distinct results; unknown identifier returns HTTP 404
Next.js frontend	Verified	Driven in a browser against the running service; success, not-found and validation paths confirmed
Evidently drift monitoring	Verified	0 of 8 drifted naturally; 6 of 8 detected under injected shift; HTML reports generated
Automated test suite	Verified	74 tests, all passing
Pipeline reproducibility	Verified	Clean-slate rerun reproduced identical metrics and predictions
Airflow orchestration	Implemented; not runtime verified	DAG structure and task order validated by parsing; scheduler execution not performed
Docker deployment	Configured; runtime verification pending	Compose definition parses and declares five services; engine never became available

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

This project designed, implemented and validated ChurnCast, an end-to-end machine learning and MLOps pipeline for subscriber churn prediction. The system covers dataset preparation, leakage-aware feature engineering, temporal evaluation, model training with probability calibration, experiment tracking, registry-based model promotion, API serving, a web interface supporting prediction by member identifier, drift monitoring, retraining orchestration and containerisation.

On the reproducible synthetic KKBOX-like prototype dataset, a calibrated Logistic Regression model selected on validation ROC-AUC of 0.7835 achieved 0.7957 ROC-AUC, 0.6904 PR-AUC, 0.7951 accuracy, 0.7309 precision, 0.4579 recall and 0.5630 F1-score on a held-out later cohort scored exactly once. The pipeline is deterministic and was verified to reproduce identical results from a clean state, and 74 automated tests cover its behaviour, including explicit assertions that post-cutoff records cannot influence any feature.

The most significant outcome of the work is methodological rather than numerical. The project began with an implementation that reported a markedly higher score, and the investigation of that score established it as a product of target leakage. Correcting the fault required rebuilding the feature layer around a per-subscriber observation cutoff and replacing random partitioning with a temporal split, and it changed the reported figure substantially downwards. The resulting system reports a lower number that describes a capability it actually possesses, which is the more useful of the two outcomes.

7.2 Key Contributions

10. A leakage-aware feature engineering layer in which the observation cutoff is a per-subscriber property, declared once, applied uniformly to every source table and asserted by automated tests rather than by convention.
11. A test suite that encodes the leakage constraint directly, adding post-cutoff records to controlled fixtures and requiring that no feature value changes.

12. A genuinely temporal evaluation protocol based on subscriber cohorts, in which model selection uses validation performance only and the held-out cohort is scored once.
13. An empirical account of a target leakage defect, including the diagnostic procedure that revealed it and the design changes it motivated, reported as a validation lesson.
14. A registry promotion gate that compares a candidate against the incumbent on validation performance and rejects candidates that do not improve, observed exercising both branches.
15. A member-identifier prediction service and web interface in which feature retrieval occurs server-side, structurally preventing divergence between training and serving feature definitions.
16. Identification and correction of two train/serve representation defects, the second of which silently disabled two categorical features at inference and was detectable only by comparing the registered model against its own reported metrics.
17. A fully reproducible pipeline, executing in approximately 60 seconds and verified to regenerate identical metrics and predictions from a clean state.

7.3 Limitations

Table 7.1 Limitations of the Present Work

No.	Limitation	Implication
1	Results are obtained on synthetic data whose churn mechanism is known by construction	Metrics validate pipeline correctness rather than estimating real-world performance; the near-equivalence of the two models is likely an artefact of the generator
2	No evaluation on the official KKBOX benchmark	No comparison against published results is possible; the preserved extracts are incomplete for this purpose
3	The preserved listening-log file is not genuine KKBOX data	Engagement features could not be validated against real listening behaviour
4	Airflow DAG not executed in a scheduler	Retry behaviour, scheduling and inter-task data passing are unverified in practice
5	Docker stack never built or started	Container build correctness and inter-service networking are unverified
6	Decision threshold fixed at 0.5	Recall of 0.4579 reflects an untuned operating point rather than a ceiling
7	No per-prediction feature attribution	The service returns a score without explaining which attributes drove it
8	No authentication on the API	Suitable for local demonstration only

7.4 Future Enhancements

Table 7.2 Future Enhancements

Priority	Enhancement	Rationale
1	Obtain the complete official KKBOX extracts and re-execute the pipeline	Would permit evaluation on real subscriber behaviour and comparison against published results; the pipeline requires only a configuration change
2	Verify the Airflow DAG in a running scheduler	Would confirm scheduling, retry behaviour and inter-task data passing, completing the orchestration objective
3	Build and start the Docker Compose stack	Would confirm container build correctness and inter-service networking
4	Tune the decision threshold against a business cost ratio	Would move the model to a defensible operating point rather than an arbitrary one, addressing the recall limitation
5	Add per-prediction feature attribution	Would allow a retention analyst to see why a subscriber was flagged, improving actionability
6	Extend monitoring with target drift and performance tracking	Would detect degradation in the relationship between features and outcome once ground truth becomes available
7	Introduce a continuous integration workflow	Would run the test suite on every change, preventing regression of the leakage guards
8	Add alerting on drift detection and promotion-gate rejection	Would convert monitoring outputs into operational signals rather than stored reports
9	Adopt a feature store abstraction	Would formalise the single-source-of-truth property currently provided by the persisted feature table
10	Deploy to a cloud environment with container orchestration	Would establish availability, scaling and rollback behaviour under realistic operating conditions

The ordering is deliberate. The first three items are verification activities that would convert components already implemented into components demonstrated, and they should precede further feature development. Items four onwards extend the system's capability once its existing behaviour has been confirmed end to end.

REFERENCES

- [1] W. Verbeke, D. Martens, C. Mues, and B. Baesens, “Building comprehensible customer churn prediction models with advanced rule induction techniques,” *Expert Systems with Applications*, vol. 38, no. 3, pp. 2354-2364, 2011.
- [2] W. Verbeke, K. Dejaeger, D. Martens, J. Hur, and B. Baesens, “New insights into churn prediction in the telecommunication sector: A profit driven data mining approach,” *European Journal of Operational Research*, vol. 218, no. 1, pp. 211-229, 2012.
- [3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.
- [5] A. Idris, M. Rizwan, and A. Khan, “Churn prediction in telecom using Random Forest and PSO based data balancing in combination with various feature selection strategies,” *Computers and Electrical Engineering*, vol. 38, no. 6, pp. 1808-1819, 2012.
- [6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [7] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189-1232, 2001.
- [8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785-794.
- [9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, USA, 2017, pp. 3146-3154.
- [10] J. C. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in *Advances in Large Margin Classifiers*, A. J. Smola, P.

Bartlett, B. Scholkopf, and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 1999, pp. 61-74.

[11] B. Zadrozny and C. Elkan, “Transforming classifier scores into accurate multiclass probability estimates,” in Proc. 8th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, Edmonton, AB, Canada, 2002, pp. 694-699.

[12] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in Proc. 22nd Int. Conf. on Machine Learning, Bonn, Germany, 2005, pp. 625-632.

[13] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, pp. 1-21, 2012.

[14] C. Bergmeir and J. M. Benitez, “On the use of cross-validation for time series predictor evaluation,” Information Sciences, vol. 191, pp. 192-213, 2012.

[15] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in Advances in Neural Information Processing Systems 28, Montreal, QC, Canada, 2015, pp. 2503-2511.

[16] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ML test score: A rubric for ML production readiness and technical debt reduction,” in Proc. IEEE Int. Conf. on Big Data, Boston, MA, USA, 2017, pp. 1123-1132.

[17] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software engineering for machine learning: A case study,” in Proc. IEEE/ACM 41st Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Montreal, QC, Canada, 2019, pp. 291-300.

[18] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, “Accelerating the machine learning lifecycle with MLflow,” IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39-45, 2018.

- [19] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1-37, 2014.
- [20] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [21] WSDM Cup 2018, "KKBOX's Churn Prediction Challenge," Kaggle, 2017. [Online]. Available: <https://www.kaggle.com/c/kkbox-churn-prediction-challenge>

APPENDICES

Appendix A - Project Directory Structure

```
ChurnCast-Prediction/
|
+-- data/
|   +-- raw/
|       +-- members.csv          synthetic prototype dataset (generated)
|       +-- transactions.csv     7,500 rows
|       +-- user_logs.csv        69,590 rows
|       +-- train.csv            519,012 rows
|       +-- cohorts.csv          7,500 rows
|       +-- cohorts.csv          7,500 rows (observation cutoffs)
|   +-- processed/
|       +-- features.csv         engineered feature table
|   +-- raw_kkbox_original/      preserved original extracts (not used)
|
+-- src/
|   +-- config.py                paths, seed, thresholds, risk banding
|   +-- pipeline.py              reusable steps shared by script and DAG
|   +-- features/features.py     leakage-aware feature engineering
|   +-- models/train.py          temporal split, training, promotion gate
|   +-- api/main.py              FastAPI service
|   +-- monitoring/drift.py      Evidently drift reporting
|   +-- data/                    original KKBOX ingestion modules
|
+-- scripts/
|   +-- generate_sample_data.py   deterministic dataset generator
|   +-- run_pipeline.py           end-to-end pipeline runner
|   +-- generate_drift_report.py  drift report generator
|
+-- pipelines/dags/
|   +-- churncast_retrain.py      Airflow retraining DAG
|
+-- frontend/
|   +-- app/page.js               Next.js web interface
|   +-- lib/api.js                member-ID prediction page
|   +-- lib/api.js                backend client
|
+-- tests/                        74 automated tests
+-- reports/                      metrics, figures, drift reports
+-- Dockerfile, docker-compose.yml, requirements.txt, README.md
```

Appendix B - API Examples

B.1 Health Check

GET /health

```
{
  "status": "ok",
  "model_available": true,
  "model_version": "1",
  "feature_table_available": true,
  "subscribers": 7500,
  "detail": null
}
```

B.2 Model Information

GET /model-info

```
{
  "model_name": "ChurnCastModel",
  "model_version": "1",
  "model_type": "logistic_regression",
  "feature_count": 27,
  "subscribers": 7500,
  "dataset": "KKBOX-like synthetic data (prototype)",
  "risk_thresholds": {"low_below": 0.4, "high_at_or_above": 0.7},
  "decision_threshold": 0.5
}
```

B.3 Member-ID Prediction

POST /predict/SYN000001

```
{
  "msno": "SYN000001",
  "churn_probability": 0.306,
  "predicted_label": 0,
  "risk_level": "LOW",
  "model_version": "1"
}
```

B.4 Unknown Member Identifier

POST /predict/SYN999999

HTTP 404

```
{
  "detail": "Subscriber SYN999999 not found"
}
```

B.5 Invocation Examples

Windows PowerShell:

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/health"
```

```
Invoke-RestMethod -Uri "http://127.0.0.1:8000/predict/SYN000001" `
-Method POST
```

curl:

```
curl http://127.0.0.1:8000/health
```

```
curl -X POST http://127.0.0.1:8000/predict/SYN000001
```

Appendix C - Configuration

All tunable values are centralised so that no threshold or path is duplicated across modules. Environment variables override the defaults, which is how the container services are configured.

```
# MLflow
MLFLOW_TRACKING_URI=sqlite:///mlflow.db
MLFLOW_EXPERIMENT=ChurnCast
MLFLOW_MODEL_NAME=ChurnCastModel

# API
CHURNCast_API_HOST=127.0.0.1
CHURNCast_API_PORT=8000
CHURNCast_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
FRONTEND_URL=http://localhost:3000

# Frontend
NEXT_PUBLIC_API_URL=http://127.0.0.1:8000

# Risk banding
CHURN_RISK_LOW_THRESHOLD=0.40      # below this -> LOW
CHURN_RISK_HIGH_THRESHOLD=0.70    # at or above this -> HIGH
CHURNCast_DECISION_THRESHOLD=0.50 # label threshold

# Data
CHURNCast_SEED=42
CHURNCast_N_SUBSCRIBERS=7500
CHURNCast_RAW_DIR=data/raw        # point at raw_kkbox_original to switch
CHURNCast_PROCESSED_DIR=data/processed
CHURNCast_REPORTS_DIR=reports
```

Appendix D - Test Results

```
$ python -m pytest tests/ -q
```

```
..... [ 97%]  
.. [100%]  
74 passed in 20.51s
```

Representative assertions from the suite:

Test	Asserts
test_post_cutoff_transactions_are_ignored	Adding a post-cutoff transaction changes no feature value
test_post_cutoff_logs_are_ignored	Adding a post-cutoff listening record changes no feature value
test_temporal_split_is_ordered_in_time	Every validation cohort is later than every training cohort, and test later still
test_features_do_not_perfectly_predict_label	No single feature separates the label, which would indicate leakage
test_registered_model_reproduces_reported_test_metrics	The registered model scores the held-out cohort exactly as the metrics document reports, to within 1e-9
test_training_and_serving_see_identical_categoricals	Categorical features are string-typed after loading, preventing encoder mismatch
test_register_if_better_rejects_a_worse_candidate	The promotion gate declines to register a candidate that does not improve
test_drift_is_detected_when_injected	An injected distributional shift is reported as drift
test_predict_unknown_msno_returns_404	An unknown identifier returns HTTP 404 with an explanatory message
test_generator_is_deterministic	The same seed produces an identical dataset

Appendix E - Sample Predictions

Produced by the registered model through the member-identifier endpoint. Identifiers SYN000001 to SYN007500 are all valid inputs.

Member ID	Probability	Label	Risk	Version
SYN000001	0.3060	0	LOW	1
SYN000002	0.4319	0	MEDIUM	1
SYN000003	0.1149	0	LOW	1
SYN000004	0.6428	1	MEDIUM	1
SYN000005	0.0857	0	LOW	1

Appendix F - Important Code Snippets

F.1 Leakage Guard in Feature Engineering

```
def _transaction_features(tx, cutoff):
    tx = tx.drop(columns=[c for c in FORBIDDEN_SOURCE_COLUMNS
                          if c in tx.columns]) # membership_expire_date
    tx = tx.merge(cutoff, on="msno", how="inner")
    tx = tx[tx["transaction_date"] <= tx["observation_cutoff"]] # LEAKAGE GUARD
    tx = tx.sort_values(["msno", "transaction_date"])
    tx["age_days"] = (tx["observation_cutoff"] - tx["transaction_date"]).dt.days
```

F.2 Temporal Split

```
def time_based_split(table):
    def part(months):
        sub = table[table["cohort_month"].isin(months)]
        return sub[FEATURE_COLUMNS], sub["is_churn"]

    X_train, y_train = part(config.TRAIN_COHORTS) # 2024-07 .. 2024-10
    X_val, y_val = part(config.VAL_COHORTS) # 2024-11
    X_test, y_test = part(config.TEST_COHORTS) # 2024-12
    return X_train, X_val, X_test, y_train, y_val, y_test
```

F.3 Nested Calibration Pipeline

```
pipe = Pipeline([("preprocess", build_preprocessor()), ("clf", estimator)])
model = CalibratedClassifierCV(pipe, method="sigmoid", cv=3)
model.fit(X_train, y_train)

# preprocessing is fitted inside each calibration fold, so no statistic from a
# validation fold can influence the transform applied to it
```

F.4 Categorical Normalisation (Train/Serve Consistency)

```
def normalise_categoricals(df):
    out = df.copy()
    for col in CATEGORICAL_FEATURES:
        s = out[col]
        numeric = pd.to_numeric(s, errors="coerce")
        if numeric.notna().any():
            # 1.0 and "1" both become "1"
            out[col] = numeric.astype("Int64").astype(str)
        else:
            out[col] = s.fillna("unknown").astype(str)
    return out
```

F.5 Member-ID Prediction Endpoint

```
@app.post("/predict/{msno}", response_model=PredictResponse)
def predict_by_msno(msno: str):
    msno = _validate_msno(msno) # 422 if empty or whitespace
    _require_ready() # 503 if model unavailable
```

```

ft = _state["features"]
if msno not in ft.index:
    raise HTTPException(404, f"Subscriber {msno} not found")

row = ft.loc[[msno]].copy()
for c in CATEGORICAL_FEATURES:
    row[c] = row[c].astype(str)
return _predict_frame(row, msno)

```

F.6 Airflow DAG Definition

```

with DAG(dag_id="churncast_retrain",
        default_args={"retries": 2,
                      "retry_delay": timedelta(minutes=2),
                      "retry_exponential_backoff": True,
                      "max_retry_delay": timedelta(minutes=15)},
        schedule="@weekly", catchup=False, max_active_runs=1) as dag:

    ingest >> build_features >> train >> evaluate >> register_if_better

```

PROGRAM OUTCOME AND PROGRAM SPECIFIC OUTCOME MAPPING

Table A.1 Program Outcome Attainment

PO	Graduate Attribute	Attained	Justification
PO 1	Engineering knowledge	Yes	Applied probability, statistics and machine learning theory to formulate subscriber churn as a supervised classification problem, and software engineering principles to construct a modular, tested and reproducible pipeline.
PO 2	Problem analysis	Yes	Identified target leakage as the dominant failure mode of the initial design through analysis of the label definition, inspection of conditional outcome distributions and a controlled window-retraction experiment.
PO 3	Design/development of solutions	Yes	Designed and implemented a leakage-aware feature layer, a temporal evaluation protocol, a calibrated two-model training pipeline, a registry promotion gate, a prediction service and a web interface.
PO 4	Conduct investigations of complex problems	Yes	Conducted controlled experiments including feature-window retraction on a fixed population and drift injection, and rejected one experimental design as confounded before drawing conclusions from it.
PO 5	Modern tool usage	Yes	Employed Python, pandas, scikit-learn, XGBoost, MLflow, FastAPI, Evidently, Next.js, pytest and Git, and prepared Airflow and Docker configurations for deployment.
PO 6	The engineer and the world	Yes	Addressed a practical retention problem in subscription services by producing calibrated risk estimates that allow finite outreach capacity to be directed where it is most effective.
PO 7	Ethics	Yes	Reported the leakage defect and its effect rather than the inflated score it produced; labelled synthetic results as synthetic; recorded a data integrity issue in the preserved extracts; and marked unverified components as unverified rather than claiming deployment.
PO 8	Individual and collaborative team work	Yes	Divided responsibility across data engineering, modelling, service development and validation while maintaining a shared version-controlled repository and a common feature contract.
PO 9	Communication	Yes	Documented the system through this report, a structured repository README recording component status, module-level docstrings explaining design decisions, and tabulated experimental results traceable to generated artifacts.
PO 10	Project management and finance	Yes	Sequenced the work so that data correctness preceded operational development, avoiding effort on orchestration and monitoring components that would have required rebuilding once the leakage defect was corrected. All tooling is open-source, incurring no licensing cost.
PO 11	Life-long learning	Yes	Acquired working knowledge of MLOps tooling, probability calibration and leakage diagnostics beyond the prescribed syllabus, and adapted the project design in response to experimental findings.

Table A.2 Program Specific Outcome Attainment

PSO	Graduate Attribute	Attained	Justification
PSO 1	Ability to design and implement intelligent systems using Artificial Intelligence, Machine Learning and Data Science techniques.	Yes	Designed and implemented a complete churn prediction system comprising leakage-aware feature engineering, calibrated Logistic Regression and gradient boosted tree models, temporal evaluation, and a registry-backed prediction service serving predictions by member identifier.
PSO 2	Ability to apply data analytics, modelling and evaluation methods to derive insights from complex real-world data and communicate them effectively.	Yes	Derived 27 behavioural features across recency, frequency, engagement, subscription and demographic categories from multi-table subscriber data; evaluated models under a temporal protocol reporting six metrics with confusion matrix and curves; and communicated both the results and their limitations in this report.
PSO 3	Ability to develop and deploy data-driven software solutions using modern tools and professional engineering practice.	Yes	Delivered a running full-stack system with a tested backend, a web interface, experiment tracking, model versioning and drift monitoring, developed under version control with automated testing and linting, and documented with an explicit distinction between verified and unverified components.

RESEARCH PAPER

ChurnCast: A Leakage-Aware MLOps Pipeline for Subscriber Churn Prediction

R SasiKirran (24AD0267) and M A Sanjay Kanth (24AD0262)

Department of Artificial Intelligence and Data Science

Chennai Institute of Technology, Chennai - 600 069

Abstract

Subscriber churn prediction in subscription services is complicated by a temporal hazard: the churn label is defined by renewal behaviour occurring after the point of prediction, so any feature computed from that period encodes the outcome rather than predicting it. We present ChurnCast, an end-to-end MLOps pipeline in which the observation cutoff is a per-subscriber property enforced in code and asserted by automated tests. We report that an earlier iteration of the system, using a single global cutoff falling inside the label period, produced a markedly inflated discriminative score, and we describe the diagnostic procedure that identified the contamination. On a reproducible synthetic dataset of 7,500 subscribers with 27 leakage-aware features and a cohort-based temporal split, a calibrated Logistic Regression model attains 0.7957 ROC-AUC and 0.6904 PR-AUC on a held-out later cohort scored once. The pipeline integrates experiment tracking, a registry promotion gate, API serving and drift monitoring, and is verified by 74 automated tests.

I. Introduction

Subscription businesses depend on retention, and the behavioural precursors of a lapse are observable before it occurs. Supervised classification over subscriber records is therefore a natural approach. The difficulty is that the label is determined after the observation point, which makes the problem unusually susceptible to target leakage. This paper describes a pipeline built around that constraint and reports the measurable consequence of violating it.

II. Method

Each subscriber is assigned an observation cutoff equal to the membership expiry about which the churn question is asked. Features are computed exclusively from records dated on

or before that cutoff; the membership expiry field itself is excluded because the label is defined against it. Twenty-seven features span recency, transaction frequency, listening frequency and volume, behavioural quality, subscription economics and lifecycle attributes.

Subscribers are grouped into monthly cohorts by cutoff, and whole cohorts are allocated to training (5,030), validation (1,235) and test (1,235) partitions, so evaluation is ordered in time. A calibrated Logistic Regression and a calibrated gradient boosted tree ensemble are trained on identical data; selection uses validation ROC-AUC only, and the test cohort is scored exactly once. Class imbalance is handled by class weighting rather than synthetic resampling, which would create records belonging to no cohort.

III. Leakage Diagnosis

The contamination in the earlier iteration was invisible to standard metrics. It was identified by two diagnostics. First, conditional churn rates computed within buckets of a candidate feature exhibited a step discontinuity aligned with the boundary of the label window, whereas a genuine behavioural feature produces a gradient. Second, a controlled experiment held the subscriber population and labels fixed while progressively retracting the feature window away from the label period, and the discriminative score declined monotonically as access was withdrawn. The monotonic decline, rather than an abrupt change, indicated a feature set steadily losing access to the label rather than a single defective column.

IV. Results

On the held-out cohort the selected model attains 0.7957 ROC-AUC, 0.6904 PR-AUC, 0.7951 accuracy, 0.7309 precision, 0.4579 recall and 0.5630 F1-score, against a cohort churn rate of 0.2883. Validation ROC-AUC was 0.7835 for the linear model and 0.7805 for the boosted ensemble. Drift monitoring reports no drift between cohorts of the same generative process and detects six of eight monitored features under a deliberately injected shift. The pipeline reproduces identical metrics from a clean state under a fixed seed.

These results correspond to a synthetic dataset constructed to mirror the structure of the KKBOX churn data. They validate the correctness and reproducibility of the pipeline and are not benchmark results on the official KKBOX dataset.

V. Discussion

Two engineering findings generalise beyond this system. First, leakage validation must be a standing test category rather than a one-time review: the defect produced no exception and no anomalous metric, and only a deliberate diagnostic revealed it. Second, representational consistency between training and serving requires explicit enforcement. A defect in which a categorical field was an integer during training and a string at inference caused the encoder to discard two features silently; it was detected only by comparing the registered model against its own reported metrics, and that comparison is now an automated test.

VI. Conclusion

Reliable churn prediction depends less on model sophistication than on disciplined temporal handling of the training data. A reproducible MLOps lifecycle, with tracked experiments, versioned models and automated leakage assertions, is what makes such discipline verifiable rather than merely intended. Future work will re-execute the pipeline on the complete official extracts, verify the orchestration and container layers at runtime, and tune the decision threshold against a business cost ratio.

References

The references supporting this paper are those listed in the References section of the accompanying project report, in particular Kaufman et al. on leakage formulation and detection, Bergmeir and Benitez on temporal validation, Niculescu-Mizil and Caruana on probability calibration, and Sculley et al. on technical debt in machine learning systems.